

Report Teamprojekt:

Machine Learning-basiertes Fahrerassistenzsystem

Nico Höhn, Philip Fritzen

Master of Science Elektrotechnik

Hochschule Darmstadt

Inhaltsverzeichnis

1 Projektdefinition.....	4
1.1 Vision-Statement.....	4
1.2 Abgrenzung.....	4
1.3 Klassisch oder Agil.....	4
1.4 Projektlaufzeit.....	4
1.5 Definition of Done (DoD) und Definition of Ready (DoR).....	5
1.6 User Stories.....	6
2 Projektplan.....	7
2.1 Arbeitspakete und Aufgabenverteilung.....	7
2.2 Zeitplan.....	8
2.2.1 Anpassungen während dem Projektverlauf.....	9
2.3 Ressourcen.....	12
2.3.1 Personal.....	12
2.3.2 Arbeitsmaterial.....	12
2.3.3 Softwaretools.....	12
2.3.4 Budget.....	13
3 Projektdurchführung.....	14
3.1 Hardwaresetup.....	14
3.1.1 Auflöten.....	15
3.1.2 Arbeitsplatz vorbereiten.....	15
3.1.3 Löten.....	16
3.1.4 Montage.....	17
3.1.5 WIFI Firmware Updaten.....	18
3.2 User Story 1 - Hilfsanwendung Fotoapparat.....	19
3.2.1 Anwendungsfall.....	19
3.2.2 Lösung.....	19
3.2.3 Bedienung.....	20
3.2.4 Entwicklungszeit und Technologiestack.....	21
3.3 User Story 5 - Gehäusekonstruktion.....	22
3.3.1 Anwendungsfall.....	22
3.3.2 Lösung.....	22
3.3.3 Zeitaufwand.....	22
3.4 User Story 2 - Trainingsdaten aufnehmen.....	23
3.4.1 Akzeptanzkriterium.....	23
3.4.2 Auflösung und Bildformat.....	23
3.4.3 Trainingsdaten-Sammelprozess.....	23
3.4.4 Schlechte Antwortzeit => Performancepatch.....	24
3.4.5 Trainingsdaten Ordnerstruktur.....	24
3.4.6 Archivierungsskript.....	24
3.4.7 Der Trainingsdatensatz.....	25
3.5 Bildvorverarbeitung in MATLAB.....	26
3.5.1 Methodischer Ablauf.....	26

3.5.2 Erkennung runde Schilder.....	26
3.5.3 Analyse der Parameterwahl bei der Kreiserkennung.....	28
3.5.4 Erkennung Dreiecke.....	31
3.5.5 Erkennung 4 Ecke.....	33
3.5.6 Fazit zur Bildvorverarbeitung in MATLAB.....	33
3.6 ML-Modell.....	35
3.6.1 Rahmenbedingungen.....	35
3.6.2 Ermittlung des Speicherbedarfs.....	36
3.6.3 Kleiner Einblick in Layerarten.....	37
3.7 User Story 6 ML-Modell(Einfluss Datenbestand auf Modellqualität).....	41
3.7.1 Erster Versuch.....	41
3.7.2 Zweiter ML Versuch.....	42
3.7.3 Dritter ML Versuch.....	42
3.8 Nutzung des Modells zum Vorsortieren der Trainingsdaten.....	43
3.9 Probleme beim Überführen von MATLAB zu OpenMV.....	44
3.9.1 Inkompatible Layer(BatchNormalizationLayer).....	44
3.9.2 Modelgröße.....	45
3.9.3 Probleme mit FC-Layer.....	45
3.10 Bildvorverarbeitung von MATLAB auf OpenMV Board bringen.....	47
3.10.1 3 Wege zum Ziel – die Alternativen.....	47
3.10.2 Ansatz: MATLAB's Code-Generator.....	47
3.10.3 Bildvorverarbeitung – der MicroPython-Weg.....	48
3.10.4 Gegenüberstellung des Zeitaufwandes.....	49
3.11 User Story 7 – ML-Modell von MATLAB auf OpenMV Board bringen.....	50
3.11.1 Die Möglichkeiten für Export und Import.....	50
3.11.2 Mit TensorFlow eine .tflite Datei erzeugen.....	51
3.11.3 Das Problem mit den Layern.....	53
4 Ergebnis.....	55
4.1 Persönlicher Erfahrungsgewinn / Lessons Learned.....	56
4.1.1 Philip.....	56
4.1.2 Nico.....	57
5 Ausblick.....	59

1 Projektdefinition

1.1 Vision-Statement

Herr Schlake ist Dozent an der Hochschule Darmstadt und unterrichtet das Wahlpflichtfach "Klassische und Machine Learning (ML) Algorithmen zur Bildverarbeitung".

Mit dem Teamprojekt: "Machine Learning-basiertes Fahrerassistenzsystem" soll mit MATLAB ein Machine Learning Modell entwickelt werden, welches anschließend auf das Mikrocontroller Board "OpenMV Cam H7 Plus" übertragen wird. Dieses soll mit einem Gehäuse, das auf ein BlueBrixx-System Fahrzeug aufgesetzt werden und dort unter verschiedenen Bedingungen BlueBrixx Verkehrszeichen erkennen.

Das Projektergebnis – vor allem die Konvertierung MATLAB Modell => OpenMV lauffähiges Modell – will er in seinen zukünftigen Vorlesungen verwenden, um den Studenten Machine Learning näherzubringen und seinem Sohn seine Arbeit zu erklären.

1.2 Abgrenzung

Das Projekt beschäftigt sich ausschließlich mit der Umsetzung eines ML-basierten Fahrerassistenzsystems zur Erkennung von Verkehrsschildern. Ein automatisiertes Fahren und die Integration in den Lehrbrief bzw. in den Unterricht ist nicht Teil dieses Projekts.

Ampeln zählen ebenfalls nicht als Verkehrsschilder und sind somit nicht Teil der Aufgabe zur automatischen Erkennung.

1.3 Klassisch oder Agil

Das Projekt gestaltet sich eher als agiles Projekt und ist aufgrund vieler unbekannter Arbeitsschritte nicht komplett klassisch planbar.

Grundlegendes Wissen zur Bildverarbeitung ist zum Projektstart zwar vorhanden, allerdings nur sehr wenig Wissen auf dem Gebiet des Machine Learning. Diese Voraussetzungen bedürfen einer agilen Vorgehensweise.

Der Zieltermin – die Prüfung – ist festgesetzt.

Das Sachziel hingegen ist nicht absolut gesetzt. Es gibt zwar Wünsche zum Projektergebnis (siehe User Stories), ob sie sich genauso umsetzen lassen, ist zum Projektstart aber offen.

1.4 Projektlaufzeit

Das Projekt startet mit einem Kick-off-Meeting am 03.04.2025.

Zeitziel und Abgabe der Projektdokumentation ist am 05.09.2025.

Der Präsentationstermin für das Projekt ist auf den 12.09.2025 datiert. Präsentationsort ist an der Hochschule Darmstadt.

1.5 Definition of Done (DoD) und Definition of Ready (DoR)

Die **Definition of Done** (DoD) legt fest, ab wann eine Aufgabe genau als abgeschlossen gilt. Wir haben für dieses Projekt folgende DoD festgeschrieben:

- Alle Akzeptanzkriterien sind erfüllt
- Ein Code-Review durch den jeweils anderen ist durchgeführt (zumindest versteht derjenige grob, was passiert)
- Falls vorhanden, sind alle für die spezifische Aufgaben definierten Testfälle bestanden
- Die Aufgabenlösung und das jeweilige Ergebnis ist in der Projektdokumentation festgehalten
- Änderungen am Code / an der Doku sind auf GitHub committed und gepushed

Die DoD setzt eine sinnvolle **Definition of Ready** (DoR) voraus, also ab wann eine Aufgabe so gut definiert ist, dass jemand daran arbeiten kann:

- Es sind Akzeptanzkriterien gemäß S.M.A.R.T.¹ Zielen oder I.N.V.E.S.T.² definiert
- Falls möglich sind Testfälle definiert, die Code oder Hardware bestehen müssen
- Die Aufgabenbeschreibung ist so verständlich geschrieben, dass wir sie beide verstehen und umsetzen könnten, auch dann, wenn nur einer diese Aufgabe übernimmt

1 [https://de.wikipedia.org/wiki/SMART_\(Projektmanagement\)](https://de.wikipedia.org/wiki/SMART_(Projektmanagement))

2 <https://agilemovement.de/agilitaet/begriffe/invest-kriterien/>

1.6 User Stories

Eine User Story ist eine Aufgabe, die ein bestimmter Benutzer in einer bestimmten Rolle mit dem Produkt erledigen will/muss. Sie entstammen dem Extreme Programming³

Die I.N.V.E.S.T Kriterien sind ein gutes Mittel, um die Qualität einer geschriebenen User Story zu bewerten.

Wir haben mit der Zustimmung von Herrn Schlake 7 User-Stories definiert:

AS A [type of user]	I NEED TO [do some task]	SO THAT I CAN [get some result]	Akzeptanzkriterium	STATUS	Anmerkung
Als Entwickler	muss ich Straße aufnehmen können	damit ich Bildmaterial für meinen Algorithmus habe	Bilder können auf SD Karte gespeichert werden	Akzeptanzkriterium erfüllt	Siehe Hilfsanwendung Fotoapparat
Als Entwickler	muss ich den Machine-Learning-Algorithmus (im folgenden ML) mit Bilddaten versorgen	damit ich den ML-Algorithmus anlernen kann	Pro Schild gibt es mindestens 30 Bilder auf mindestens 3 verschiedenen Hintergründen. Das Auto wurde zwischen den Fotos langsam auf das Schild in normaler Fahrtrichtung zu bewegt.	Akzeptanzkriterium erfüllt	
Als Entwickler	möchte ich an das Display ein Foto übertragen wenn auf dem Foto min. ein Schild erkannt wurde. Das erkannte Schild soll mit einem weißen Rahmen markiert werden,	damit ich erkennen kann, wie gut meine Schilderkennung funktioniert	Das aktuellste Foto, auf dem ein Schild erkannt wurde, wird auf dem Display angezeigt. Um alle als Schild erkannte Bereiche wird ein weißer Rahmen gelegt.	Wurde aus Zeitmangel nicht umgesetzt	
Als Entwickler und Dozent	möchte ich die Wahrscheinlichkeit, mit der das richtige Schild erkannt wurde ablesen können	damit ich einschätzen kann wie gut der ML-Code das Schild erkennt	Die Wahrscheinlichkeit wird zusätzlich auf dem Display angezeigt	Wurde aus Zeitmangel nicht umgesetzt	
Als Dozent	möchte ich ein Auto mit montierter Kamera haben	damit ich ein Simulationsfahrzeug habe	Kameramodul auf Legofahrzeug befestigt. Legofahrzeug kann verschoben werden ohne, dass das Modul herunterfallen kann.	Akzeptanzkriterium erfüllt	Gehäuse für Board inklusive BlueBrixx Befestigung ist angefertigt
Als Lego-Autofahrer	möchte ich ein Fahrerassistenzsystem, dass Schilder erkennen kann	damit ich auf die Schilder hingewiesen werde	Das Schild wird mit x% Wahrscheinlichkeit erkannt.	Akzeptanzkriterium erfüllt MATLAB erreicht 99% bei den Validierungsdaten Für OpenMV gibt es noch keinen Messwert	Erreichbare Wahrscheinlichkeit wird während dem Projektfortschritt ermittelt
Als Dozent	möchte ich ein Fahrerassistenzsystem auf ML-Basis, dass Schilder erkennen kann	damit ich ein Beispiel für meine Vorlesungen habe	Die Vorgehensweise um einen Matlab ML-Algorithmus zu erstellen und mit einem OpenMV Board zu verwenden ist dokumentiert.	Akzeptanzkriterium erfüllt	

Aus diesen User Stories haben wir über eine Work-Breakdown-Structure Aufgabenpakete abgeleitet und festgehalten.

³ <http://www.extremeprogramming.org/rules/userstories.html>

2 Projektplan

2.1 Arbeitspakete und Aufgabenverteilung

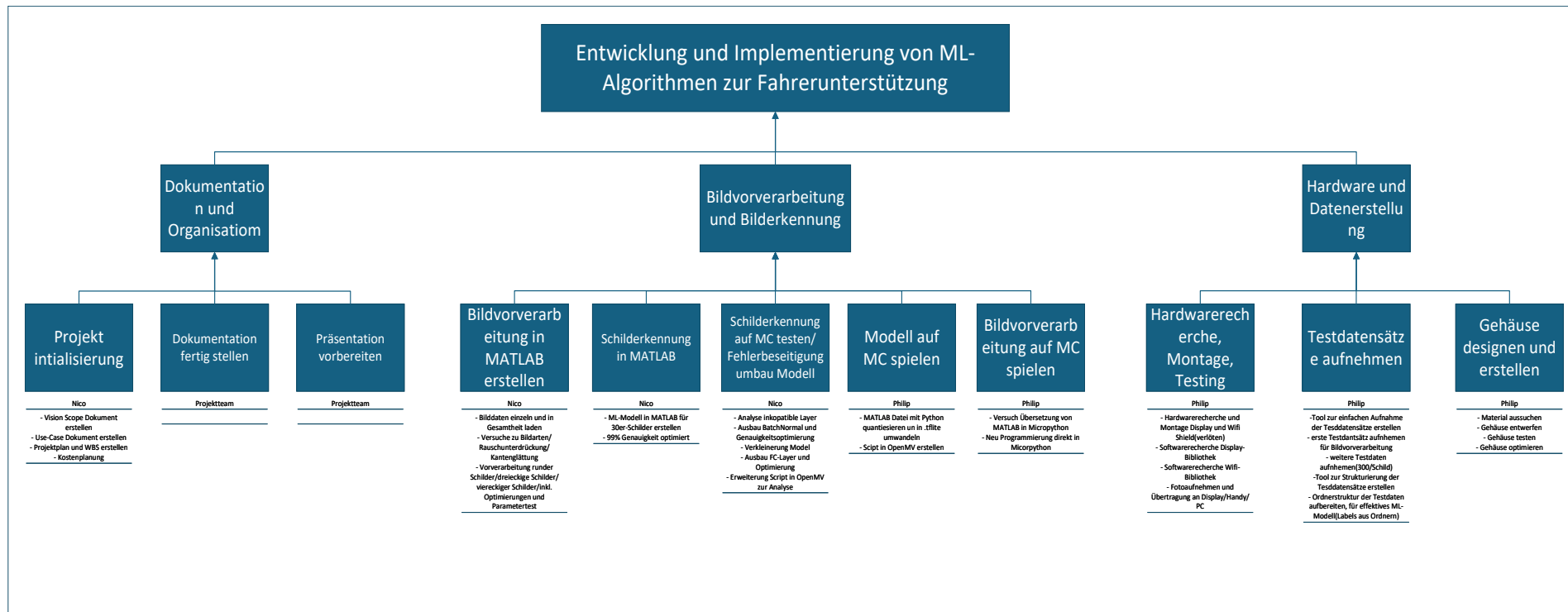


Abbildung 1: Work Breakdown Structure

2.2 Zeitplan

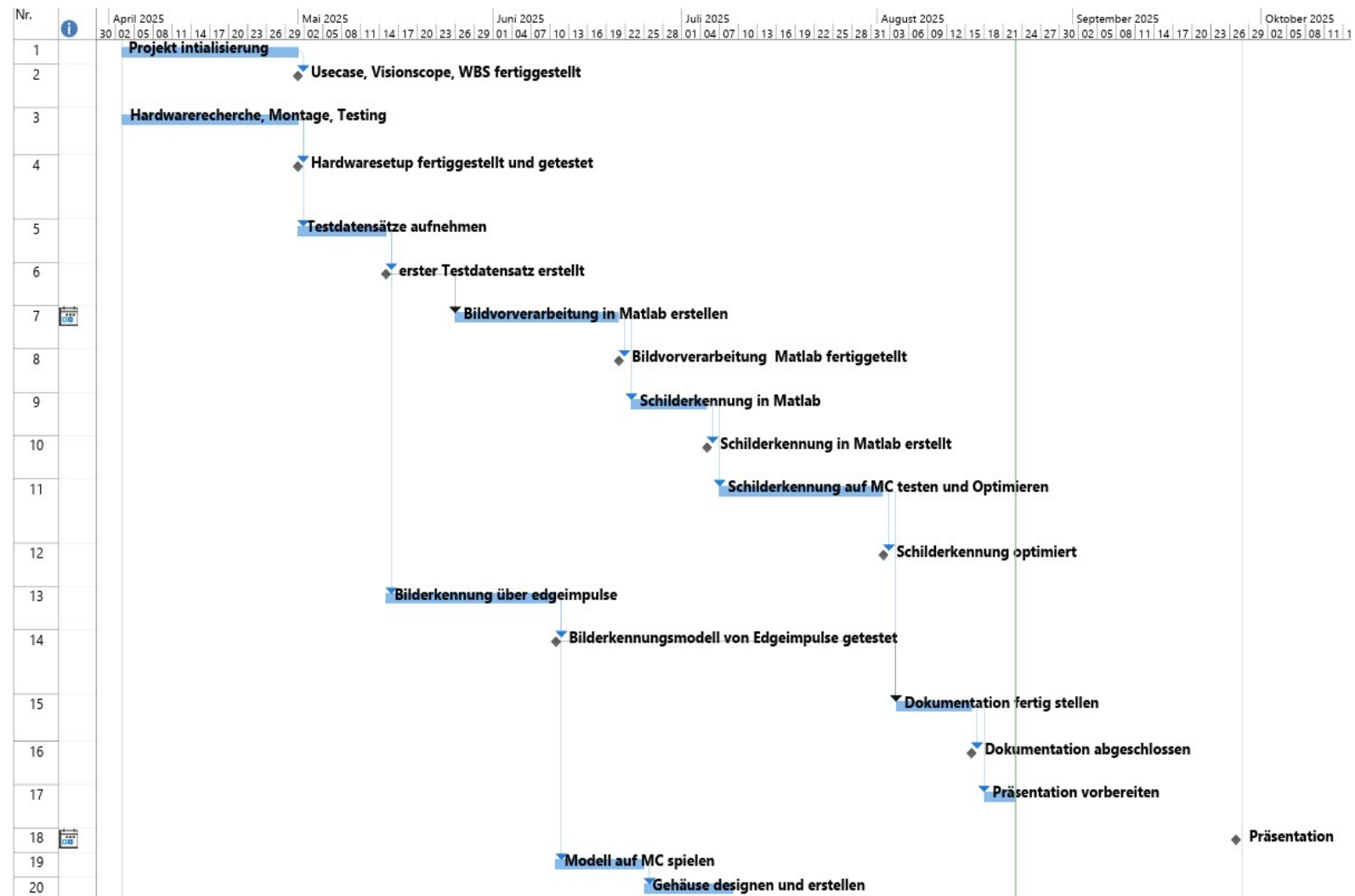


Abbildung 2: Initialer Projektplan

2.2.1 Anpassungen während dem Projektverlauf

Im folgenden ein Protokoll der Veränderungen am geplanten Arbeitsablauf. Im Abschnitt 3 Projektdurchführung greifen wir die hier genannten Punkte in den zugehörigen Kapiteln noch einmal auf.

Verlängern von „Projektinitialisierung“ von 4 auf 6 Wochen

Während der Projektinitialisierung stellen wir fest, dass der ursprünglich angesetzte Zeitraum von vier Wochen nicht ausreichen wird, um die erforderlichen Arbeitspakete in der gewünschten Qualität zu finalisieren. Insbesondere die Ausarbeitung der User Stories, des Vision Scopes sowie des Work Breakdown Structures (WBS) nimmt mehr Zeit in Anspruch als zunächst angenommen.

Ursprünglich war geplant, in dieser Phase lediglich erste Entwürfe zu erstellen. Im weiteren Verlauf entscheiden wir jedoch, dass wir die Projektinitialisierung mit einem abgestimmten und vom Kunden abgenommenen Abschluss beenden wollen. Dies bedeutet, dass sämtliche Inhalte mit dem Kunden besprochen, angepasst und final freigegeben werden. Durch diesen erweiterten Anspruch an die Ergebnisqualität ist eine Verlängerung der Initialisierungsphase auf insgesamt sechs Wochen erforderlich.

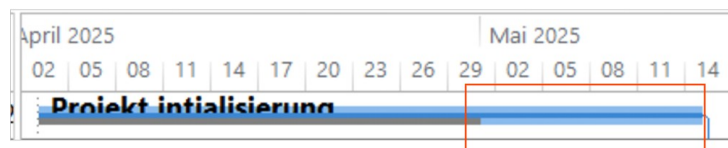


Abbildung 3: Verlängerung der Projektinitialisierung

Vorziehen von „Gehäuse designen und erstellen“

Beim Aufnehmen der ersten Trainingsdaten erkennen wir, dass die Aufnahme von Testbildern ohne ein physisches Gehäuse mit erheblichen praktischen Einschränkungen verbunden ist. Die fehlende Stabilität führt zu Problemen bei der Positionierung der Kamera.

Aus diesem Grund ziehen wir die Erstellung des Gehäuse-Designs zeitlich vor.

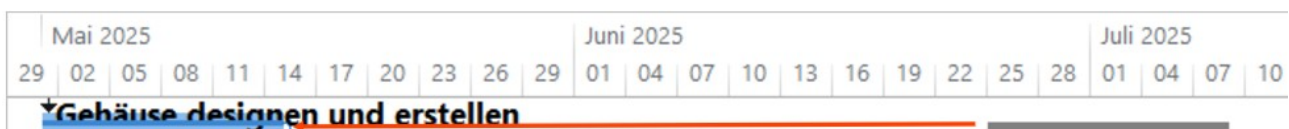


Abbildung 4: Vorziehen von Gehäusedesign

Verschieben und unterteilen von „Testdatensätze aufnehmen“

Im Verlauf der Projektarbeit zeigt sich, dass das Sammeln der Trainingsdaten aufwändiger ist als ursprünglich angenommen. Das Aufsuchen unterschiedlicher Orte, aufbauen, Aufnehmen der Bilder inklusive Überschuss, abbauen und aussortieren ist ein erheblicher Aufwand. Dieser erhöhte Aufwand macht eine Verlängerung der Testdatensatzaufnahme notwendig(1).

Um die daraus resultierende Zeitverzögerung zu minimieren, beginnen wir parallel mit der Entwicklung der Bildvorverarbeitung in MATLAB(2) und nutzen dafür die ersten fertig kategorisierten Bilder.

Außerdem spalten wir den Meilenstein für eine bessere Verfolgung der Fortschritte in „ersten Testdatensatz erstellen“ und „weitere Testdaten sammeln“ auf(3).

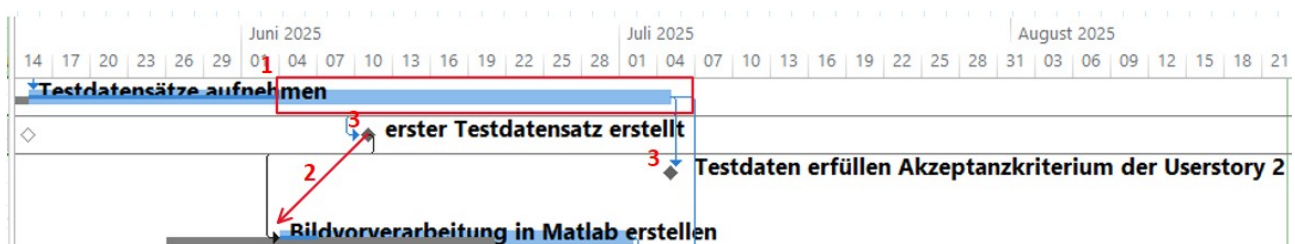


Abbildung 5: Verlängerung „Testdatensätze aufnehmen“ und Unterteilung Meilenstein

Vorziehen von „Modell auf μ C spielen“ und streichen von „Bilderkennung über Edgeimpulse“

Auf Grundlage des geänderten Projektplans und dem recht frühen Prüfungstermin im September schätzen wir, dass wir die ursprünglich vorgesehenen Aufgaben nicht alle realisieren können. In Abstimmung mit Herrn Schlake priorisieren wir daher das Aufspielen des Modells auf den Controller und streichen die Bilderkennung über Edge Impulse aus dem Projektumfang.



Abbildung 6: Modell auf MC spielen (Vorher)

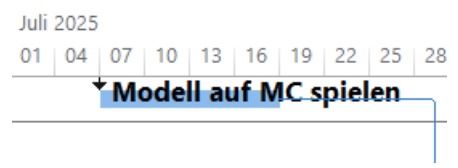


Abbildung 7: Modell auf MC spielen (Nacher)

**„Schilderkennung auf MC testen und Optimieren“ anpassen zu
„Schilderkennung auf MC testen/Fehlerbeseitigung Umbau Modell“**

Aufgrund verschiedener auftretender Probleme beim Versuch, das Modell auf dem Mikrocontroller lauffähig zu machen, sehen wir uns gezwungen, den ursprünglichen Arbeitsschritt „Schilderkennung auf MC testen und optimieren“ in „Schilderkennung auf MC testen / Fehlerbeseitigung / Umbau des Modells“ umzubenennen. Dadurch wird der tatsächliche Entwicklungsaufwand angemessen abgebildet.

Der Meilenstein „Machine-Learning Modell lässt sich auf dem Board ausführen“ hängt dadurch sowohl vom Vorgang “Modell auf MC spielen” als auch von „Schilderkennung auf MC testen/Fehlerbeseitigung Umbau Modell“ ab.



Abbildung 8: Fehlerbeseitigung

2.3 Ressourcen

2.3.1 Personal

- Nico Höhn
- Philip Fritzen

2.3.2 Arbeitsmaterial

- Ein Laptop / PC / Smartphone
- Ein OpenMV Cam H7 Plus Mikrocontroller Board
 - Inklusive Display, 2 Akkus und einem WIFI-Modul
- Ein mobiler WLAN-Router mit Powerbank
- Zum Löten:
 - eine Lötkolbenstation
 - Löthalterung (dritte Hand)
 - Lötrauchabsaugung
 - Lötzinn
 - Entlötlitze
 - Flussmittel
 - Isopropylalkohol
 - Reinigungsstäbchen
- Prusa MK3S+ 3D Filamentdrucker
 - Filament: „PETG CF Red“ von „The Filament“

2.3.3 Softwaretools

- OpenMV IDE – <https://openmv.io/pages/download>
- Netron – <https://netron.app>
- Python 3.11.11
 - TensorFlow 2.19.0 – <https://www.tensorflow.org>
 - Pillow 11.3.0 – <https://pillow.readthedocs.io/en/stable/index.html>
- 3D-Modellier FreeCAD – <https://www.freecad.org>
- LibreOffice – <https://de.libreoffice.org>
- Visual Studio Code – <https://code.visualstudio.com>

- git inklusive GitHub – <https://github.com/Commander-Philip/the-machine-learning-brick>
- MATLAB – <https://matlab.mathworks.com>
- ChatGPT als Suchmaschine – <https://chatgpt.com>
- ProjectLibre – <http://projectlibre.de>
- DrawIO – <https://www.drawio.com>
- PlantUML – <https://github.com/plantuml/plantuml>
- Microsoft OneNote – <https://www.microsoft.com/de-de/microsoft-365/onenote/digital-note-taking-app>
- Microsoft Project – <https://www.microsoft.com/de-de/microsoft-365/project/project-management>
- Microsoft Visio – <https://www.microsoft.com/de-de/microsoft-365/visio/visio-in-microsoft-365>

2.3.4 Budget

Bezeichnung	Preis
Akku+Micro SD	35,79 €
OpenMV Cam, WifiShield,LCD Shield	189,21 €
Bluebrixx	29,60 €
Filament	32,78 €
Versandkosten Gehäuse	9,58 €
Gesamtkosten	287,38 €

Tabelle 1: Budgetrechnung

3 Projektdurchführung

3.1 Hardwaresetup

Um ein Machine Learning Modell zu erstellen, schauen wir uns zuerst die Hardware an, auf dem das Modell laufen soll.

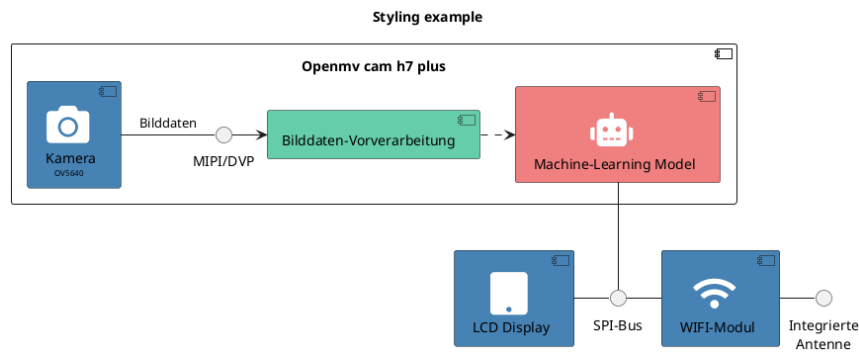


Abbildung 9: Prinzipaufbau des Mikrocontroller-Board inklusive Peripherie

Das Hardwaresetup besteht aus den folgenden Komponenten:

- Das „OpenMV Cam H7 Plus“ Board (im weiteren einfach Cam-Board genannt) mit aufgesteckter Kamera und eingesteckter SD-Karte
- Das LCD Display „OMV-LCD“
- Das WIFI-Modul „OMV-WIFI“
- Einem 3,7V Akku für die „Standalone“ Stromversorgung

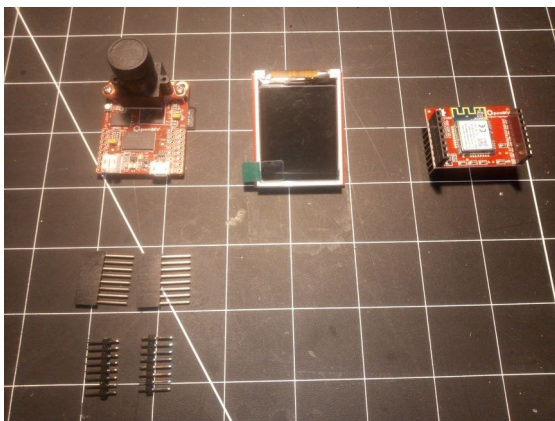


Abbildung 10: Einzelkomponenten Vorderseite

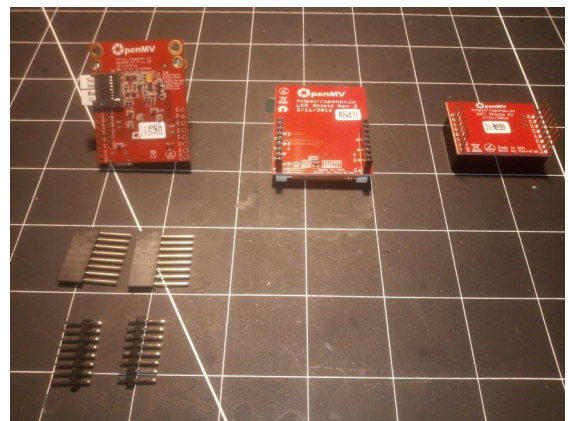


Abbildung 11: Einzelkomponenten Rückseite

Wie wir auf den Bildern erkennen können, wird das Mikrocontroller Board selbst mit unverlöteten Buchsen- und Stiftleisten geliefert. Das Display und das WIFI Modul werden dagegen vorgelötet geliefert.

3.1.1 Auflöten

Da wir das Display und das WIFI-Modul verwenden wollen, müssen die Buchsenleisten in die THT-Bohrungen gelötet werden. Der Auflötvorgang wird im folgenden Beschrieben.

Buchsenleistenposition

Um die Buchsenleiste zu verlöten, stecken wir die Header auf die Stifte des WIFI-Moduls und führen anschließend die Stifte der Buchsenleiste in die vorgesehenen THT-Bohrungen des Cam-Boards ein. Damit ist die Steckverbindung beim Verlöten schon richtig ausgerichtet und das WIFI Modul wie auch das Display werden sich später beim aufstecken und abziehen nicht verkanten.

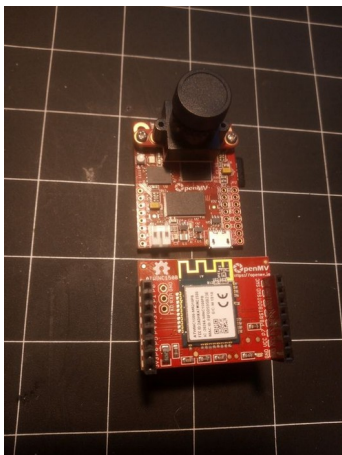


Abbildung 12: Leisten 1

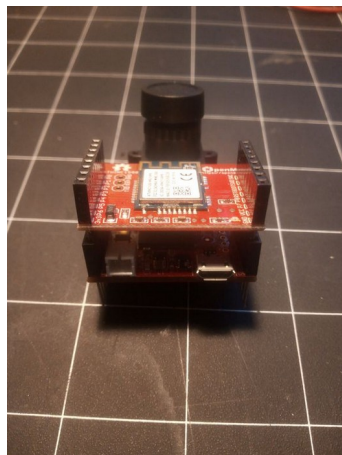


Abbildung 13: Leisten 2



Abbildung 14: Leisten 3

3.1.2 Arbeitsplatz vorbereiten

Nun können wir das Board in eine Löthilfe einspannen werden und den restlichen Arbeitsplatz vorbereiten.



Abbildung 15: Arbeitsplatz zum Löten



Abbildung 16: Arbeitsplatz Nahaufnahme

Zu sehen sind hier die Löthalterung, der LötKolben und eine Lötrauchabsaugung, welche etwas erhöht steht, damit der Rauch nicht darüber hinweg zieht.

Als weiteres Hilfsmittel legen wir Lötzinn, Entlötlitze und Flussmittel bereit.

ULF 10

EDSYN ZINKOHLSPRAY

Entzündend

Schutzhandschuhe und Schutzkleidung

Char. n.: 340205.2 Inhalt: 250 ml

EDSYN GmbH EUROPA

Finkenweg 2, D-97892 Krauthausen

[illegible]

3.1.4 Montage

Die fertig verlöteten Buchsenleisten sehen so aus:

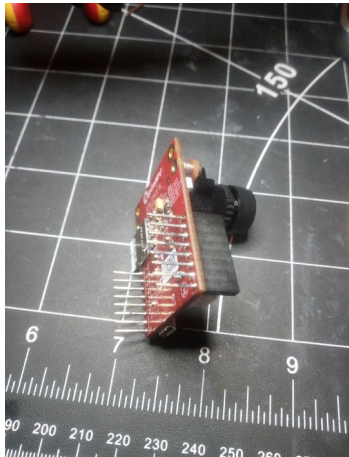


Abbildung 21: Verlötete Buchsenleiste rechts

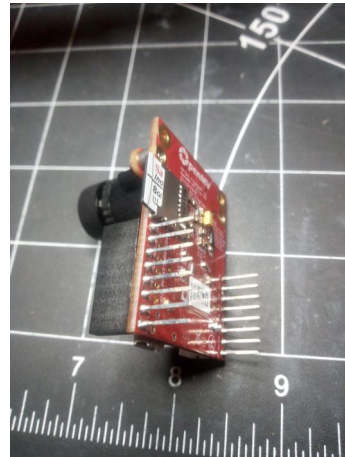


Abbildung 22: Verlötete Buchsenleiste links

Nun können wir Display und WIFI-Modul einfach aufstecken.

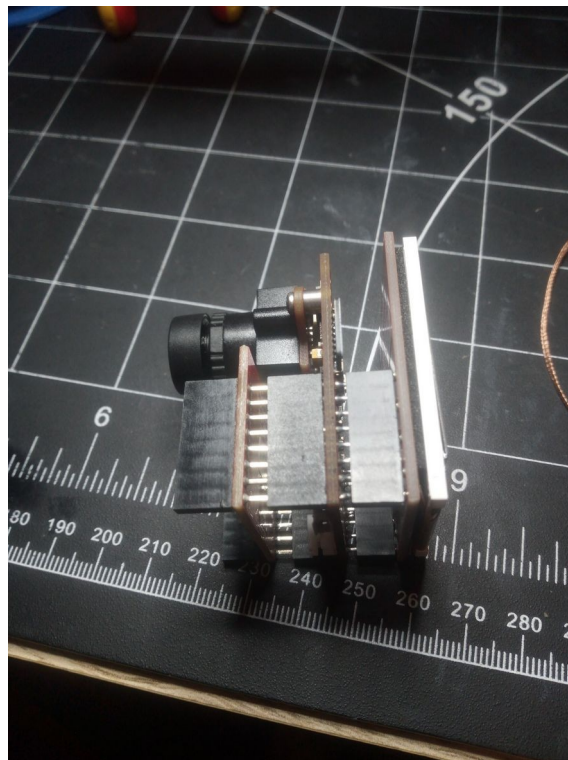


Abbildung 23: OpenMV µC Board mit WIFI und Display Modul

3.1.5 WIFI Firmware Updaten

Bevor wir das Cam-Board in Betrieb nehmen, müssen wir die Firmware updaten. Es folgt eine Beschreibung der Vorgehensweise

1. Wir ziehen das Display zur Sicherheit vom Board ab, damit nur ein Teilnehmer am SPI Bus hängt
 - Interessanterweise nutzen Display und WIFI-Modul ohne weitere Modifikation den gleichen Slave-Select Pin des SPI Busses, obwohl laut SPI Definition jeder Slave einen eigenen haben soll.
2. Das Cam-Board schließen wird über das Micro-USB Kabel (nicht im Lieferumfang enthalten) an den PC an
3. Dann öffnen wir die OpenMV IDE starten und verbinden über den Knopf unten links das Board



Abbildung 24:
Connect

4. Das `fw_update.py` Skript finden wir unter
File => Examples => WiFi => WINC1500 => `fw_update.py`
 - Der verbaute WIFI Mikrochip auf dem vorliegenden Cam-Board ist der ATWINC1500-MR210PB
5. Die SD-Karte binden wir mit einem Dateimanager einbinden und wie in den Code-Kommentaren beschrieben kopieren wir die Firmware aus dem Ordner `<openmv-ide-install-dir>/share/qtcreator/firmware/WINC1500/winc_19_7_6.bin` (Linux installation) auf die SD-Karte, ohne sie in einen Unterordner zu legen.
6. Wir werfen die SD-Karte über den Dateimanager aus
7. Über Tools => Reset OpenMV Cam starten wir das Cam-Board neu und drücken wieder den Knopf aus Schritt 3
8. Im Code bei Zeile 23, im Methodenaufruf `wlan.fw_update` entfernen wir den führenden Schrägstrich aus dem Parameterpfad, da sonst das Skript die Datei nicht finden kann.
9. Wir führen den Code über den Startknopf unten links aus

3.2 User Story 1 - Hilfsanwendung Fotoapparat

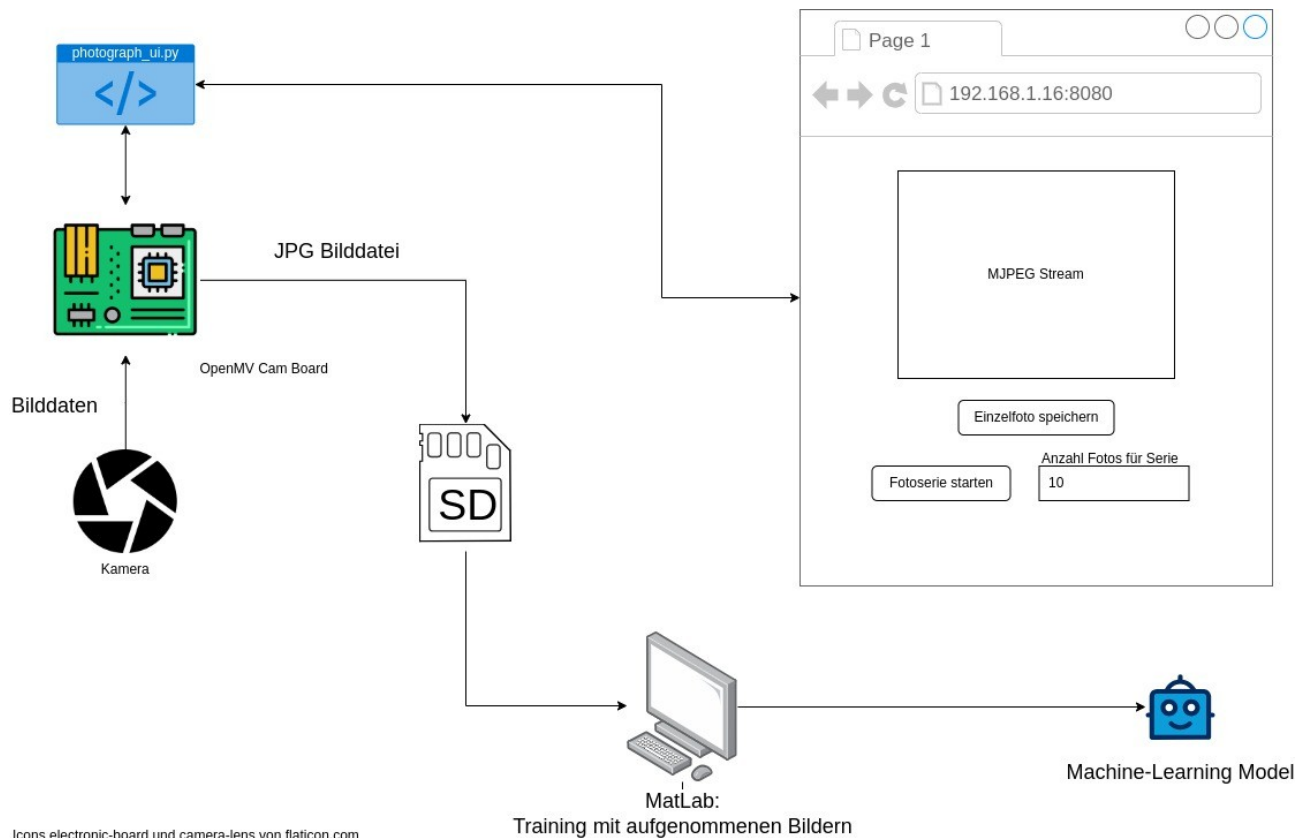


Abbildung 25: Konzept des Trainings

3.2.1 Anwendungsfall

Um den Machine Learning Algorithmus für die Schilderkennung zu trainieren brauchen wir viele Bilder. Sinnvollerweise sollten diese Bilder in der Qualität aufgenommen werden, mit dem im Produktivbetrieb die Bilder zyklisch aufgenommen und bewertet werden.

Deshalb nehmen wir die Bilder mit dem OpenMV Board selbst auf und speichern sie auf der SD-Karte.

3.2.2 Lösung

Um dies möglichst bequem und für viele Bilder auch schnell zu erledigen, entscheiden wir uns für die Entwicklung einer Hilfsanwendung. Das Programm, welches auf dem Cam-Board läuft, verbindet sich mit einem konfigurierten WLAN-Hotspot. Dies kann auch ein Smartphone Hotspot sein.

Nachdem wir die IP-Adresse des Boards aus dem Log des Programms entnommen haben, kann die Oberfläche mit einem Browser über die URL „http://<ip-adresse-des-boards>:8080/“ (ohne Anführungszeichen) aufgerufen werden.

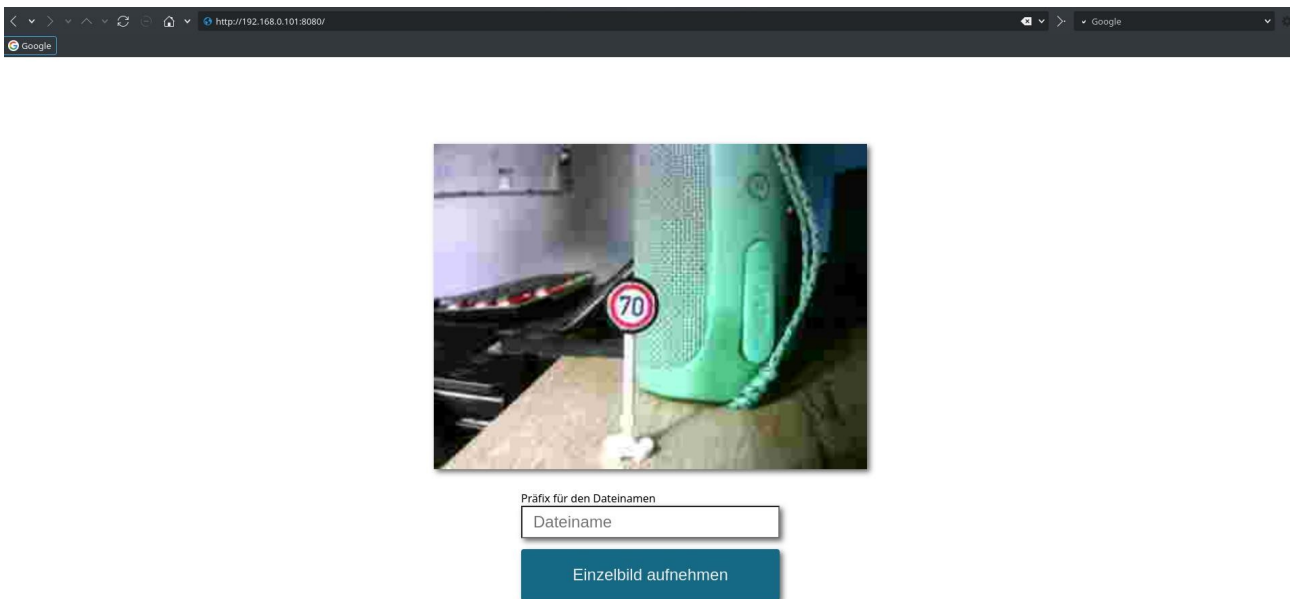


Abbildung 26: Aufruf des Fotoapparats über die Adresse <http://192.168.0.101:8080>

Das Board sendet darauf hin eine Webseite zurück, in der ein Live-Feed der Kamera in geringer Qualität zu sehen ist. Außerdem gibt es dort einen Button „Einzelbild aufnehmen“ und ein Eingabefeld für einen Dateinamen.

Die Anwendung ist – samt einer Anleitung zum installieren auf dem Cam-Board – unter <https://github.com/Commander-Philip/the-machine-learning-brick/tree/main/Hilfsanwendung%20Fotoapparat> zu finden.

3.2.3 Bedienung

Über den Live-Feed können wir die Kamera des Boards so ausrichten, wie wir das Foto aufnehmen möchten. Den Dateinamen anzugeben ist optional. Drücken wir nun den Button „Einzelbild aufnehmen“, speichert das Board ein Foto in der Auflösung 1280 x 720 Pixeln direkt auf der eingelegten SD-Karte im Ordner „/pictures“.

Die Dateien werden im Format „<dateiname>-<datum>-<fortlaufende Nummer>.jpg“ benannt.

✓ Foto Geschwindigkeitsbegrenzung_70-2025-08-24_6.jpg erfolgreich gespeichert

✗ Foto speichern fehlgeschlagen!

Abbildung 27: Erfolgs- und Fehlermeldung bei der Bildaufnahme

Ob die Aufnahme gelang oder nicht, wird durch eine Meldung in der oberen rechten Ecke angezeigt.

3.2.4 Entwicklungszeit und Technologiestack

Die Entwicklung der Hilfsanwendung dauerte inklusive Dokumentation 4 Personenwochen.

Wir verwenden in der Anwendung die Technologien HTML, CSS, Javascript und MicroPython.

3.3 User Story 5 - Gehäusekonstruktion

3.3.1 Anwendungsfall

Beim Versuch, die ersten Trainingsdaten ohne Halterung für das Board aufzunehmen, ist uns aufgefallen, dass das Ausrichten der Kamera sehr Umständlich ist und zu lange dauern würde.

Deshalb haben wir die Konstruktion des Gehäuses für das Cam-Board vorgezogen.

3.3.2 Lösung

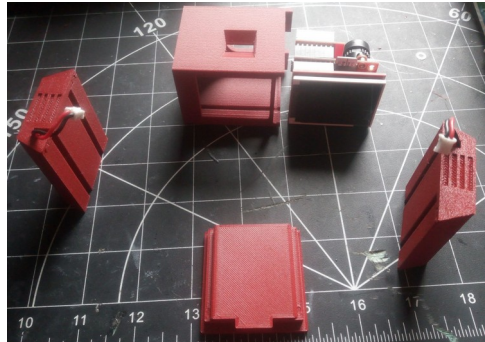


Abbildung 28: Gehäuse in Einzelteilen



Abbildung 29: Gehäuse montiert von links



Abbildung 30: Gehäuse montiert von rechts

Das Gehäuse besteht aus einem Korpus für das Board selbst, einem Akkupack und einer Verbindungsplatte, die mit dem BlueBrixx Klemmsystem kompatibel ist.

Das Akkupack wird über zwei Schwalbenschwänze mit dem Korpus verbunden, das Cam-Board wird mit der Leiterplatte in zwei Nuten im Korpus eingeführt und mit der BlueBrixx-Verbindungsplatte als Deckel gesichert.

Das Gehäuse ist somit komplett ohne Werkzeug und Schrauben zu montieren sowie demontieren.

3.3.3 Zeitaufwand

Das Design und der Druck des Gehäuses nahm 2 Personenwochen und ca. 4 Design Iterationen in Anspruch.

3.4 User Story 2 - Trainingsdaten aufnehmen

Im folgenden gehen wir nun daran, mit der in Kapitel 3.2 beschriebenen Hilfsanwendung Trainingsdaten aufzunehmen. Die Bilder sollen im nächsten Schritt dazu dienen, eine Bildvorverarbeitung aufzusetzen und das Machine Learning Model zu trainieren.

3.4.1 Akzeptanzkriterium

Das Akzeptanzkriterium für die User-Story laute, pro Schildart mindestens je 30 Bilder auf mindestens 3 unterschiedlichen Hintergründen aufzunehmen.

Im zur Verfügung gestellten BlueBrixx Set sind 12 Schildarten enthalten.

3.4.2 Auflösung und Bildformat

Als Auflösung für die Aufnahmen und den späteren Produktionseinsatz wählen wir 1280x720 Pixel. Dies entspricht im MicroPython Code der Konstanten `sensor . HD`.

Als Bildformat kommt RGB565 zum Einsatz, welches die Farbe eines Pixels mit 16 Bit⁴ darstellt. Jeweils 5 Bit für Rot und Blau, sowie 6 Bit für Grün.

3.4.3 Trainingsdaten-Sammelprozess

Der Aufnahmeprozess sieht wie folgt aus:

1. Eine passende Umgebung für den Hintergrund suchen
2. BlueBrixx Straße aufbauen
3. Mobilen WLAN-Router an Powerbank anschließen
4. BlueBrixx Auto mit Cam-Board bestücken und betriebsbereit machen
5. Hilfsanwendung „Fotoapparat“ im Browser von Smartphone oder Laptop öffnen
6. Ein Schild auf der Straße befestigen
7. Mindestens 100 Fotos von dem befestigten Schild machen
 - Zwischen den Fotos das Auto langsam auf das Schild zu bewegen
8. Das befestigte Schild gegen das nächste zu fotografierende Schild austauschen
9. Die Schritte 7 und 8 wiederholen, bis für alle Schilder mindestens 100 Bilder existieren
10. Bilder von SD-Karte des Cam-Boards auf PC übertragen und unbrauchbare Bilder aussortieren
11. Mit Schritt 1 erneut beginnen

Da die Bildaufnahme und -aufbereitung recht Zeitaufwändig ist, nehmen wir uns für das Sammeln von Trainingsdaten nach der Entwicklung der Hilfsanwendung weitere 4 Wochen Zeit.

4 <https://rgbcolorpicker.com/565>

3.4.4 Schlechte Antwortzeit => Performancepatch

Beim Aufnehmen der Bilder ist uns aufgefallen, dass die Antwortzeit des Boards mit jedem Schild länger dauert. Nach einer Problemanalyse ist uns aufgefallen, dass der Code zum Nummerieren der Dateien bei jedem neuen Foto einmal alle bereits vorhandenen Dateien scannt.

Um dies zu verbessern, geben wir der Hilfsanwendung einen kleinen Patch. Die Bilder werden nun pro angegebenen Dateinamen in einem Ordner zusammengefasst und nur beim Ändern des Dateinamens in der Browseranwendung werden die Dateien neu gescannt. Dies sorgt dafür, dass die gefühlte Antwortzeit des Boards für ein Foto gleich bleibt.

3.4.5 Trainingsdaten Ordnerstruktur

Die Bilder legen wir in einer bewusst gewählten Ordnerstruktur ab, um sie schnell einsortieren zu können:

Ordner mit Schildform => Ordner mit Schildart => Ordner mit Ort => Bilddatei

Hier zwei Beispielpfade für die Bilder der Schilder

„Geschwindigkeitsbegrenzung_70km/h,, und „Vorfahrt_Achten“:

1_Runde_Schilder/Geschwindigkeitsbegrenzung_70/Ackerstraße/
Geschwindigkeitsbegrenzung_70-2025-06-12__1.jpg

2_Dreieckige_Schilder/Vorfahrt_Achten/Bushaltestelle/Vorfahrt_Achten-2025-06-05__2.jpg

3.4.6 Archivierungsskript

Um die Bilder möglichst einfach zwischen den Teammitgliedern auszutauschen und das ML-Model möglichst leicht mit Trainingsdaten versorgen zu können, haben wir das Python-Skript „prepare-upload.py“⁵ geschrieben, welches den Ordner mit dem Ortsnamen entfernt und die Bilder in das Archiv „Trainingsdaten.zip“ mit der Ordnerstruktur

Ordner mit Schildform => Ordner mit Schildart => Bilddatei

verpackt.

So konnten während dem Sammeln der Trainingsdaten iterativ weitere Bilder ohne manuellen Aufwand neu gepackt und ausgetauscht werden.

Außerdem erfasst das Skript die Anzahl der archivierten Bilder pro Schild, wie im nächsten Abschnitt zu sehen ist.

5 <https://github.com/Commander-Philip/the-machine-learning-brick/blob/main/Trainingsdaten/prepare-upload.py>

3.4.7 Der Trainingsdatensatz

Als Ergebnis dieser Aufgabe liegt uns folgende Ausgabe aus dem „prepare-upload.py“ Skript vor:

```
Für ./1_Runde_Schilder/Geschwindigkeitsbegrenzung_30 gibt es 310 Bilder
Für ./1_Runde_Schilder/Geschwindigkeitsempfehlung_90 gibt es 382 Bilder
Für ./1_Runde_Schilder/Durchfahrt_Verboten_Einbahnstrasse gibt es 373 Bilder
Für ./1_Runde_Schilder/Geschwindigkeitsbegrenzung_70 gibt es 356 Bilder
Für ./1_Runde_Schilder/Geschwindigkeitsbegrenzung_100 gibt es 372 Bilder
Für ./1_Runde_Schilder/Geschwindigkeitsbegrenzung_50 gibt es 284 Bilder
Für ./1_Runde_Schilder/Geschwindigkeitsempfehlung_30 gibt es 375 Bilder
Für ./5_Polygon_Schilder/Stoppschild gibt es 372 Bilder
Für ./4_Ampel/Ampel_Rot gibt es 290 Bilder
Für ./4_Ampel/Ampel_Gruen gibt es 311 Bilder
Für ./4_Ampel/Ampel_Gelb gibt es 277 Bilder
Für ./3_Rechteckige_Schilder/Verkehrsberuhigter_Bereich gibt es 329 Bilder
Für ./3_Rechteckige_Schilder/Fussgaengerueberweg gibt es 323 Bilder
Für ./2_Dreieckige_Schilder/Achtung_Gefahrenstelle gibt es 301 Bilder
Für ./2_Dreieckige_Schilder/Vorfahrt_Achten gibt es 396 Bilder
Für ./6_Challenge/Geschwindigkeitsbegrenzung_30_und_Fussgaengerueberweg gibt es 39 Bilder

Insgesamt 5090 Bilder gezippt
```

3.5 Bildvorverarbeitung in MATLAB

Im Anschluss an die Zusammenstellung und Beschreibung des Trainingsdatensatzes widmen wir uns nun dem Einlesen und der Vorverarbeitung der Bilddaten. Dieser Schritt bildet die Grundlage für alle weiteren Verarbeitungsschritte und ist entscheidend für die Qualität der späteren Erkennung.

3.5.1 Methodischer Ablauf

MATLAB Live Scripts eignen sich hervorragend für das Prototyping, da sie eine unmittelbare Visualisierung der Ergebnisse ermöglichen. So lässt sich direkt nachvollziehen, welcher Code welche Wirkung erzielt.

- **Einlesen der Bilddaten:** Für unsere Versuchsreihen und Funktionstests lesen wir zunächst einzelne Bilder mit `imread` ein, z.B. um gezielt Grauwerte zu ermitteln und zu sehen, was bei einzelnen Funktionen genau passiert. Nachdem wir die Einzelbildverarbeitung erfolgreich getestet haben, greifen wir für die eigentliche Vorverarbeitung auf den gesamten Bildbestand über `Imagestore` zu. Über `Labelsource` können aus dem Ordnernamen auch gleich die Label für die spätere Erkennung gespeichert werden.

```
% Bild laden
I = imread(imdsSchilder.Files{i});
%Bilder des gesamten ordners in ImagedataStore speichern
% und aus den Ordnernamen die Label erstellen
imdsSchilder = imageDatastore(directory, "IncludeSubfolders", true, "LabelSource", "foldernames");
```

Abbildung 31: MATLAB – Bilder einlesen

- **Formklassifikation:** Wir kategorisieren die vorhandenen Schilder nach ihrer Grundform: rund, dreieckig, rechteckig und polygonal.
- **Farbinformationen:** Farben dienen als zusätzliche Merkmale zur Verbesserung der Vorverarbeitung. Die Kombinationen sind
 - o Runde Schilder – Blau und Rot
 - o Dreieckige Schilder – Rot
 - o Rechteckige Schilder – Blau
 - o Polygonförmige Schilder – Rot
- **Markieren:** Zur Durchführung der Analyse markieren wir den entsprechenden Bildabschnitt visuell.
- **Ausschneiden:** Anschließend erfolgt die Generierung eines definierten Bildausschnitts, der das erkannte Objekt gezielt umfasst.

3.5.2 Erkennung runde Schilder

Wir untersuchen, welche Bildarten Farbbilder, Graustufenbilder oder Binärbilder sich am besten für die Verarbeitung eignen und wie sich die Wahl der Bildart auf die Speichergröße

auswirkt. Außerdem prüfen wir die Auswirkungen von Rauschunterdrückung und Kantenglättung auf die Bildqualität und die nachfolgenden Verarbeitungsschritte.

Vergleich Farb-, Graustufen- und Binärbilder

```
grayImg = rgb2gray(img);           % In Grau konvertieren
binaryImage = imbinarize(grayImage); % Binärbild erzeugen
grayImagefilter = imgaussfilt(grayImage, 2); % Rauschentfernung
edges = edge(grayImage, 'Sobel');   % Kantenerkennung mit Sobel

imshow(I);
[centers, radii] = imfindcircles(I, [15, 50], 'Sensitivity', 0.9)
viscircles(centers, radii, 'EdgeColor', 'b');
imshow(grayImage);
[centers, radii] = imfindcircles(grayImage, [15, 50], 'Sensitivity', 0.9)
viscircles(centers, radii, 'EdgeColor', 'b');
imshow(grayImagefilter);
[centers, radii] = imfindcircles(grayImagefilter, [15, 50], 'Sensitivity', 0.9)
viscircles(centers, radii, 'EdgeColor', 'b');
imshow(binaryImage);
[centers, radii] = imfindcircles(binaryImage, [15, 50], 'Sensitivity', 0.9)
viscircles(centers, radii, 'EdgeColor', 'b');
```

Abbildung 32: MATLAB – Erkennung runde Schilder

Speichergröße

- Wir wandeln die Bilder in Graubilder bzw. Binärbilder um. Die Speichergröße verändert sich dadurch nicht, auch Kantenglättung und Rauschunterdrückung verändern die Speichergröße nicht.

Genauigkeit

- Weder die Umwandlung in Graubilder/Binärbilder noch die Anwendung von Kantenglättung oder Rauschunterdrückung verbessern die Formerkennung. Im Gegenteil, sie wirken sich negativ auf die Ergebnisqualität aus. Auch weitere Optimierungsversuche führen zu keiner Verbesserung.
- Die Anzahl der gefundenen Kreise hat sich durch die vorherige Umwandlung in ein Graubild nicht verändert. Dies liegt daran, dass die Funktion `imfindcircles` Farbbilder automatisch in Graubilder umwandelt.

Für unsere Schilderkennung werden wir daher keine Binärisierung, Kantenglättung oder Rauschunterdrückung anwenden.

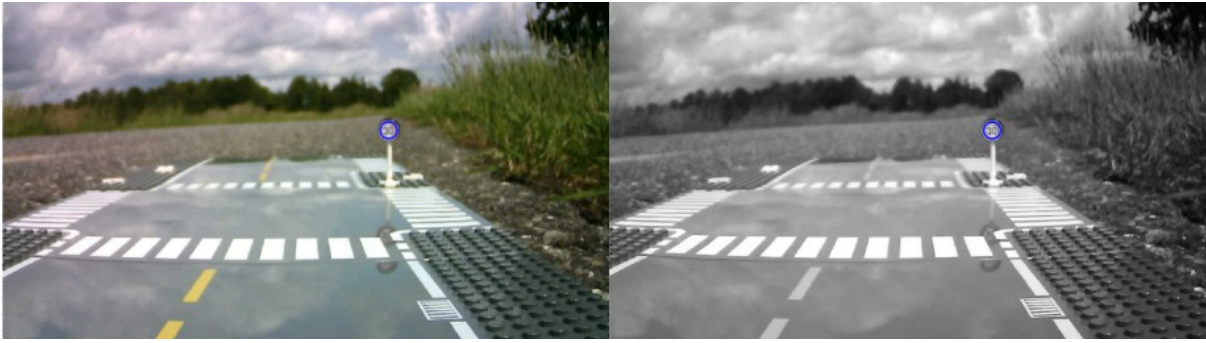


Abbildung 33: Kreiserkennung – Farb- und Grau-bild

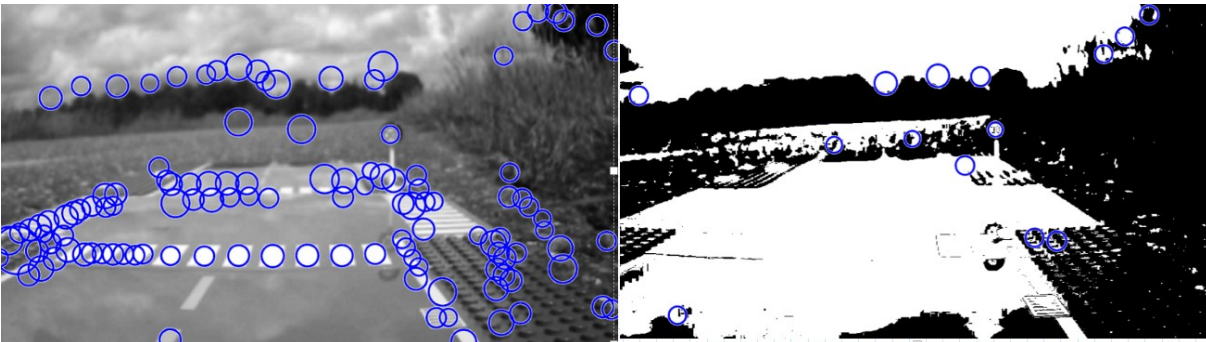


Abbildung 34: Kreiserkennung – Rauschunterdrückung und Binärbild

3.5.3 Analyse der Parameterwahl bei der Kreiserkennung

Im Rahmen der Bildverarbeitung untersuchen wir, wie sich unterschiedliche Einstellungen der Parameter Radius und „Sensitivity“ auf die Erkennung runder Verkehrszeichen auswirken.

Dabei stellen wir fest, dass die Wahl des Radius maßgeblich beeinflusst, ob zu kleine oder zu große Kreise erkannt werden. Ein zu klein gewählter minimaler Radius führt dazu, dass eine Vielzahl von Kreisen detektiert wird, darunter auch viele irrelevante Strukturen.

Die Anpassung der „Sensitivity“ zeigt ebenfalls deutliche Auswirkungen. Eine hohe „Sensitivity“ führt zur Erkennung zahlreicher Kreise, darunter auch viele falsch-positive Ergebnisse. Ist die Sensitivität hingegen zu niedrig eingestellt, werden kaum noch Kreise erkannt.

```
imshow(I);
[centers, radii] = imfindcircles(I, [42, 70], 'Sensitivity', 0.99)
viscircles(centers, radii, 'EdgeColor', 'b');
```

Abbildung 35: Kreiserkennung – Sensitivity

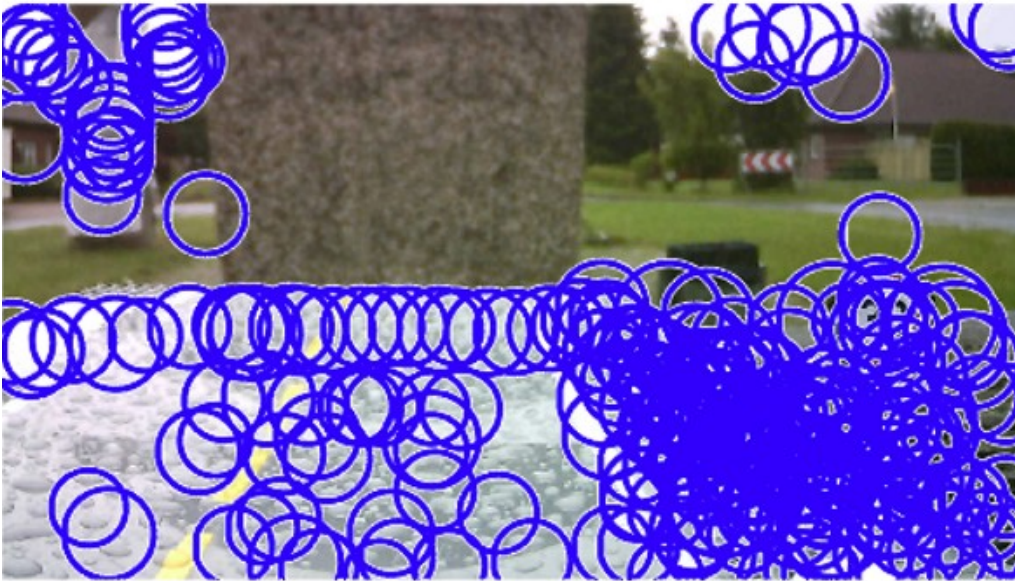


Abbildung 36: Beispielbild – Hohe Sensitivität

Bei unserer Analyse fällt uns auf, dass insbesondere die blauen Verkehrszeichen Schwierigkeiten bei der Erkennung bereiten. Im Gegensatz dazu erkennen wir rot umrandete Schilder wie die Geschwindigkeitsbegrenzungen und „Verbot der Einfahrt“ zuverlässig und vollständig mit der Anwendung von `imfindcircles`⁶⁷. Eine „Sensitivity“ im Bereich von 0,90 bis 0,92 erweist sich dabei als optimal.

```
[centers, radii] = imfindcircles(I, [20, 200], 'Sensitivity', 0.9);
```

Abbildung 37: MATLAB-Code Sensitivität



Abbildung 38: Erkennung rote runde Schilder

Problemstellung bei blauen runden Schildern

Bei der Erkennung von runden Schildern kommen wir mit der Verwendung von `imfindcircles` nicht weiter.

Nur auf einem von zehn Fotos erkennen wir das Schild als Kreis. Selbst bei geringer Distanz zum Schild wird es nicht als Kreis erkannt.

6 <https://www.mathworks.com/help/releases/R2024b/images/ref/imfindcircles.html>

7 <https://www.mathworks.com/help/releases/R2024b/images/ref/viscircles.html>



Abbildung 39: Blau Schilder

Auch unsere Testreihen mit den verschiedenen Parametern zeigen keine Verbesserungen. Beispiel Erhöhung der Sensitivität und des Radius.

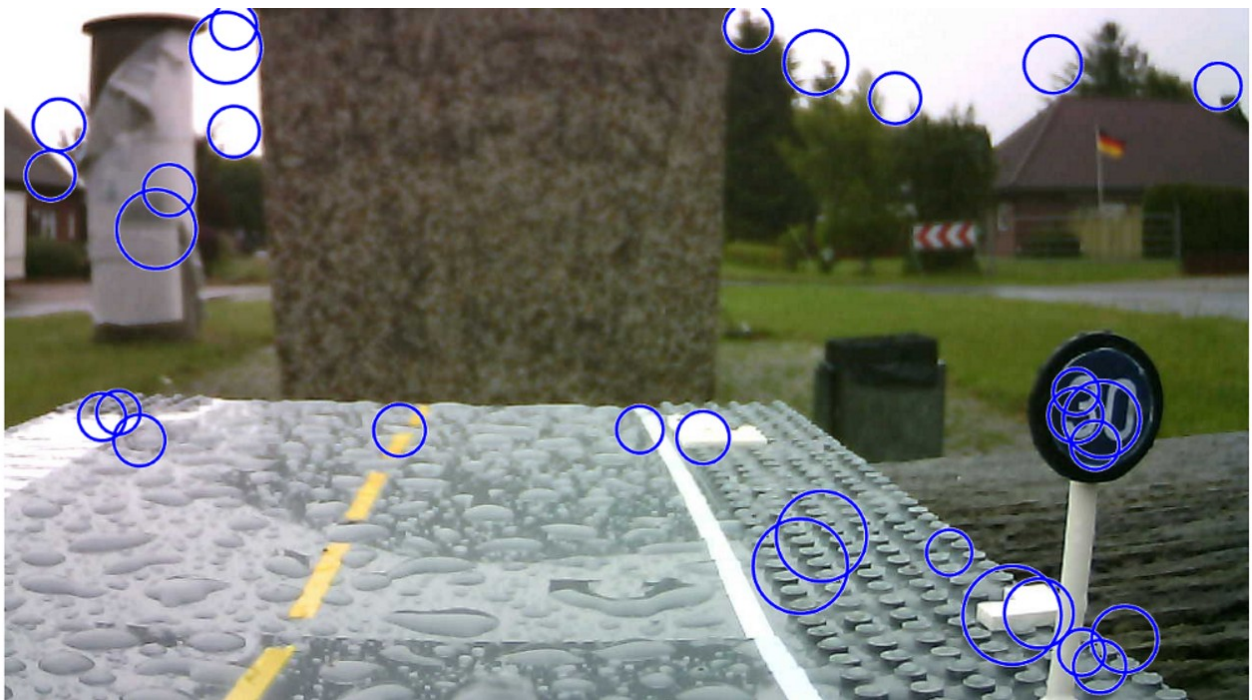


Abbildung 40: Blaue Schilder Fehlererkennung

Bilder gesamt		7	12	
				30 Ackerstraße Kreisegesamt
Radius	Sensity	90 Bushalte Schildkreisgefunden	30 Ackerstraße	Kreisegesamt
20-200	0,9	1	2	3
20-201	0,91	3	5	8
20-202	0,92	3	8	16
20-203	0,93	3	8	54
20-204	0,94	2,5	8	122

Abbildung 41: Parametervergleich

Auch unsere Versuche mit dem Parameter `ObjectPolarity` helfen uns nicht weiter. Je nachdem, ob das Bild insgesamt dunkler oder heller ist, führen die Einstellungen „dark“ bzw. „bright“ zu besseren Ergebnissen.

Bilder gesamt			7	12		10
Radius	Sensity	ObjectPolarity'	90 Bushalte Schildkreisgefun den	30 Ackerstraße	30 Ackerstraße Kreisegesamt	30 Bushaltestelle
20-200	0,92	bright	3	8	17	6
20-200	0,92	dark	4	4	17	9

Abbildung 42: Parametervergleich 2

Lösung

Da die Verkehrszeichen einen schwarzen Rand aufweisen, wurde zur Verbesserung der Erkennung ein Binärbild mit einem Schwellenwert von 40 erzeugt. Diese gezielte Schwellenwertsetzung führte zu einer deutlichen Steigerung der Erkennungsgenauigkeit.

Bilder gesamt			7			12(ein Schild pixelgröße 19)				10		
			90 Bushalte Schildkreisgefun den	Gesamtanzahl gefundenkreise	Gesamtanzahl gefundenkreise	30 Ackerstraße	bemerkung	30 Ackerstraße Kreisegesamt	30 Bushaltestelle	bemerkung	Gesamtanzahl gefundenkreise	
Radius	Sensity	Binärbild										
20-200	0,92	25				11		60				
20-200	0,92	15				8		15				
20-200	0,92	20				10		25				
20-200	0,92	30				10						
20-200	0,9	25	4			11	alle außer den kleinsten	18	3			
20-200	0,9	40	7	18		9	3 der 4 kleinsten erkennt er nicht	58	8	alle außer die 2 kleinsten	25	

Abbildung 43: Parametervergleich 3 erfolgreich

Damit schließen wir auch die Bildvorverarbeitung für runde blaue Schilder ab.

3.5.4 Erkennung Dreiecke

Da es für die Erkennung von Dreiecken keine vordefinierte Funktion gibt, gehen wir hier neue Wege. Mit der Funktion `regionprops`⁸ extrahieren wir verschiedene Eigenschaften aus einem Bild. Sie misst Merkmale wie Fläche, Schwerpunkt und Bounding Box für jedes Objekt (also jede verbundene Komponente).

`Regionprops` identifiziert eindeutige Objekte in Binärbildern mithilfe der 8 benachbarten Pixel. Da `regionprops` binäre Bilder benötigt, bestimmen wir den Schwellenwert automatisch über `graythresh` und erzeugen daraus die entsprechenden Binärbilder.

⁸ <https://www.mathworks.com/help/releases/R2024b/images/ref/regionprops.html>


```

grayImage = rgb2gray(I);
thresh = graythresh(grayImage); % automatische Schwellenwertbestimmung
C = imbinarize(grayImage, thresh); % Schwarz-Weiß-Bild

% Objekte labeln und Eigenschaften analysieren, Achtung hintergrund
% pixel(Schwarz) werden als Hintergrund nicht als Objekt erkannt
E = bwlabel(C); % Labeling der Objekte
stats2 = regionprops(C, 'Area', 'Perimeter', 'BoundingBox', 'ConvexHull', 'Extrema');

```

Abbildung 44: MATLAB-Code 3eck Erkennung

Nach vielen Versuchen zeigt sich, dass die Eigenschaft `ConvexHull` für die Dreieckerkennung am vielversprechendsten ist. `ConvexHull` liefert das kleinste Polygon, das eine Region vollständig umschließt, dargestellt als eine $p \times 2$ -Matrix, wobei jede Zeile die x- und y-Koordinaten eines Scheitelpunkts enthält⁹.

Damit nur die Hauptpunkte (also die Ecken, die die Form definieren) erkannt werden, vereinfachen wir das Polygon mithilfe von `reducepoly`¹⁰. Da der erste Punkt in der Matrix doppelt vorkommt, hat die Matrix 4 Einträge bei 3 Ecken, was wir bei der Erkennung berücksichtigen.

```

epsilon = 0.5;
for i = 1:length(stats2)
    hull = stats2(i).ConvexHull;
    %Nur große genug Objekte weiterverarbeiten
    if stats2(i).Area > 500
        % Vereinfachung des Polygons, so das nur die Hauptpunkte die das Polygon
        % ausmachen gefunden werden
        simplified = reducepoly(hull, epsilon);
        numVertices = size(simplified, 1);
        if numVertices == 4

```

Abbildung 45: MATLAB-Dreieckerkennung



Abbildung 46: 3eck Erkennung

Damit schließen wir auch die Bildvorverarbeitung für Dreieckige-Schilder ab.

⁹ <https://de.mathworks.com/help/releases/R2024b/images/ref/regionprops.html>

¹⁰ https://de.mathworks.com/help/releases/R2024b/images/ref/reducepoly.html?s_tid=doc_ta

3.5.5 Erkennung 4 Ecke

Da die Schilder ebenfalls blau sind, gehen wir ähnlich vor wie bei den runden blauen Schildern und verwenden eine manuelle Schwelle von 40 zur Erstellung der Binärbilder. Da `regionprops`¹¹ mit den Vordergrundpixeln arbeitet, müssen wir das Bild zuvor invertieren.

```
D = ~C; % Invertierung des Bildes
```

Abbildung 47: MATLAB_Code Bild Invertierung

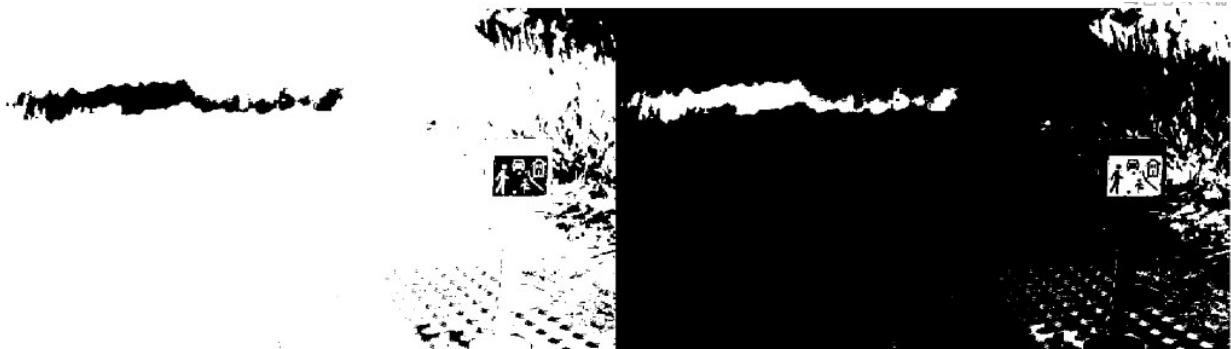


Abbildung 48: Bild Invertierung

Nun suchen wir ein geeignetes Epsilon. Für das Beispielbild erweist sich der Wert 0,06 als optimal, es ist der höchste Wert, bei dem ausschließlich das Schild als Viereck erkannt wird.

```
% Toleranz für Eckenvereinfachung  
epsilon = 0.06;
```

Abbildung 49: MATLAB-Code Toleranz blaue Vierecke



Abbildung 50: blaue Vierecke

Die Erkennung ist noch nicht optimal, aber aus zeitlichen Gründen können wir die Viereck-Erkennung, in dem Projekt, nicht weiter verbessern.

3.5.6 Fazit zur Bildvorverarbeitung in MATLAB

Wir haben erfolgreich Verfahren zur Vorverarbeitung aller betrachteten Schildformen entwickelt. Bei Kreisen und Dreiecken liegen die Ergebnisse bereits auf einem sehr hohen

¹¹ <https://de.mathworks.com/help/releases/R2024b/images/ref/regionprops.html>

Niveau. Für rechteckige Formen konnte ein tragfähiges Grundkonzept erstellt werden, das weiteren Verfeinerungen bedurft.

Bemerkenswert war, dass wir auch ohne vertiefte Vorkenntnisse und ohne externe Unterstützung robuste Ergebnisse erzielen konnten. Das zeigt, dass sich Bildvorverarbeitung selbstständig erlernen lässt.

Gleichzeitig hat die, in späteren Kapiteln beschriebene, Portierung der Bildvorverarbeitung vom MATLAB-Prototyp auf das OpenMV-Board gezeigt, dass sich in MATLAB vor allem ein funktionierendes Grundkonzept ausarbeiten lässt. Detaillierte Parameteroptimierungen und aufwändige Untersuchungen sollten hingegen direkt auf der Zielhardware durchgeführt werden.

3.6 ML-Modell

3.6.1 Rahmenbedingungen

Modellformat und Quantisierung

- **Modellformat:** Es wird ausschließlich das TensorFlow Lite-Format (.tflite) unterstützt. Andere Modelltypen oder dynamische Graphstrukturen sind nicht kompatibel und können nicht ausgeführt werden.
- **Quantisierung:** Nur Modelle mit 8-Bit-Quantisierung (int8/uint8) sind lauffähig. Modelle mit Gleitkommazahlen (Float) sind nicht lauffähig.
- **Was das bedeutet:** Damit das Modell trotzdem gute Ergebnisse liefert, muss man es entweder schon beim Training auf die 8-bit-Nutzung vorbereiten oder es nachträglich sehr sorgfältig anpassen – am besten mit echten Beispieldaten zur Kalibrierung.

Speicherbudgets und Layerkompatibilität

- **Modellgröße:** OpenMV H7 Plus/RT1062 $\approx$ 4–8 MB (durch SDRAM/Heap-Erweiterung)
- **Laufzeit-Heap:** Neben den Gewichten müssen auch Aktivierungstensoren, Framebuffer, temporäre Arbeitsbereiche und Post-Processing in den verfügbaren Heap passen.
- **Operatoren:** Nur TFLite-Mikro-Operatoren sind zulässig; exotische Layer (z. B. komplexe Attention-Blöcke, benutzerdefinierte Aktivierungen) sind nicht verfügbar.
- **Was das bedeutet:** Man sollte möglichst einfache und bekannte Bausteine wie Conv2D, Pooling, ReLU oder MaxPooling verwenden
 - **Problem dabei:** Es gibt keine Liste, welche Layer wirklich erlaubt sind und die Informationen, die man findet, widersprechen sich teilweise.

Funktionale Rahmenbedingungen

- **Kein Training auf der Zielhardware:** Modellupdates erfolgen in Matlab/Python und werden als neue .tflite-Modelle aufgespielt.

3.6.2 Ermittlung des Speicherbedarfs

Formel: Modellgröße (in Bytes) = Anzahl der Parameter $\times$ Größe pro Parameter

In MATLAB bestimmen wir die Anzahl der Parameter eines Netzwerks mit der Funktion `analyzeNetwork(layers)`¹². Der Eintrag „Total Learnables“ zeigt uns dabei, wie viele Parameter im Modell enthalten sind.

81.4k
total learnables

Abbildung 51:
Total learnables

Den größten Einfluss im Beispiel auf den Speicherbedarf hat Layer 10 FC. Von insgesamt 81400 Parameter stammen 80000 von ihm.

LAYER INFORMATION				
	Name	Type	Activations	Learnable Sizes
1	imageinput 200×200×3 images with 'zerocenter' normaliza...	Image Input	200(S) $\times$ 200(S) $\times$ 3(C) $\times$ 1(B)	-
2	conv_1 8 3×3×3 convolutions with stride [1 1] and pad...	2-D Convolution	200(S) $\times$ 200(S) $\times$ 8(C) $\times$ 1(B)	Weights 3 $\times$ 3 $\times$ 3 $\times$ 8 Bias 1 $\times$ 1 $\times$ 8
3	batchnorm_1 Batch normalization with 8 channels	Batch Normalization	200(S) $\times$ 200(S) $\times$ 8(C) $\times$ 1(B)	Offset 1 $\times$ 1 $\times$ 8 Scale 1 $\times$ 1 $\times$ 8
4	relu_1 ReLU	ReLU	200(S) $\times$ 200(S) $\times$ 8(C) $\times$ 1(B)	-
5	maxpool_1 2×2 max pooling with stride [2 2] and padding ...	2-D Max Pooling	100(S) $\times$ 100(S) $\times$ 8(C) $\times$ 1(B)	-
6	conv_2 16 3×3×8 convolutions with stride [1 1] and pa...	2-D Convolution	100(S) $\times$ 100(S) $\times$ 16(C) $\times$ 1(B)	Weights 3 $\times$ 3 $\times$ 8 $\times$ 16 Bias 1 $\times$ 1 $\times$ 16
7	batchnorm_2 Batch normalization with 16 channels	Batch Normalization	100(S) $\times$ 100(S) $\times$ 16(C) $\times$ 1(B)	Offset 1 $\times$ 1 $\times$ 16 Scale 1 $\times$ 1 $\times$ 16
8	relu_2 ReLU	ReLU	100(S) $\times$ 100(S) $\times$ 16(C) $\times$ 1(B)	-
9	maxpool_2 2×2 max pooling with stride [2 2] and padding ...	2-D Max Pooling	50(S) $\times$ 50(S) $\times$ 16(C) $\times$ 1(B)	-
10	fc 2 fully connected layer	Fully Connected	1(S) $\times$ 1(S) $\times$ 2(C) $\times$ 1(B)	Weights 2 $\times$ 40000 Bias 2 $\times$ 1
11	softmax softmax	Softmax	1(S) $\times$ 1(S) $\times$ 2(C) $\times$ 1(B)	-
12	classoutput crossentropy with classes 'BilderOhneSchilde...	Classification Output	1(S) $\times$ 1(S) $\times$ 2(C) $\times$ 1(B)	-

Abbildung 52: Layeransicht MALAB

Erläuterung Gewichte FC-Layer

Im oberen Beispiel erhält der Fully Connected Layer seine Eingabe aus dem letzten Pooling-Layer mit einer Aktivierungsgröße von $50 \times 50 \times 16 = 40.000$ Neuronen. Bei einer vollständigen Verbindung zu 2 Ausgabeneuronen ergibt sich eine Gesamtanzahl von 80.000 Parametern.

Da jeder zusätzliche Klassenausgang ein weiteres Neuron im Fully ConnectedLayer erfordert, steigt der Speicherbedarf linear und für jede weitere Klasse werden weitere 40000 Parameter benötigt.

¹² https://de.mathworks.com/help/releases/R2024b/deeplearning/ref/analyzenetwork.html?searchHighlight=analyzeNetwork&s_tid=doc_srchtile

3.6.3 Kleiner Einblick in Layerarten

Convolutional Layer

Ziel:

- Erstellung der Feature-Map
- Feature-Map zeigt, wo ein bestimmtes Merkmal im Bild vorkommt
- Gibt die Intensität des Merkmals an jeder Position an

Funktionsweise:

- Ein kleiner Filter (z. B. 3×3 Matrix) wird über das Eingabebild geschoben
- Der Filter multipliziert seine Werte mit den entsprechenden Pixeln im Bildausschnitt und summiert das Ergebnis das ergibt einen neuen Pixelwert
- Ergebnis: eine neue Matrix die *Feature Map*

Filtergröße

- Definiert den Bildbereich, den ein Neuron „sieht“
- Größere Filter (z. B. 5×5 oder 7×7) → größere rezeptive Fläche
- Vorteil: Erkennung größerer oder komplexerer Muster¹³¹⁴

FC-Layer

Ziel:

- gelernte Merkmale aus den vorherigen Schichten zu einer finalen Entscheidung oder Vorhersage zu kombinieren

Funktionsweise:

- Jedes Neuron ist mit jedem Neuron des vorherigen Layers verbunden
- Jeder Eingangswert wird mit einem Gewicht multipliziert
- Alle Ergebnisse werden aufsummiert
- Dann kommt oft noch ein Bias-Wert dazu

Nicht positionssensitiv:

- Im Gegensatz zu Convolutional Layers, keine räumliche Struktur mehr
- Wandelt die extrahierten Features in Klassenzuordnungen, Scores oder Regressionswerte um¹⁵

13 <https://www.claimflow.de/post/was-ist-ein-convolutional-neural-network>

14 https://de.wikipedia.org/wiki/Convolutional_Neural_Network

15 <https://www.geeksforgeeks.org/deep-learning/what-is-fully-connected-layer-in-deep-learning/>

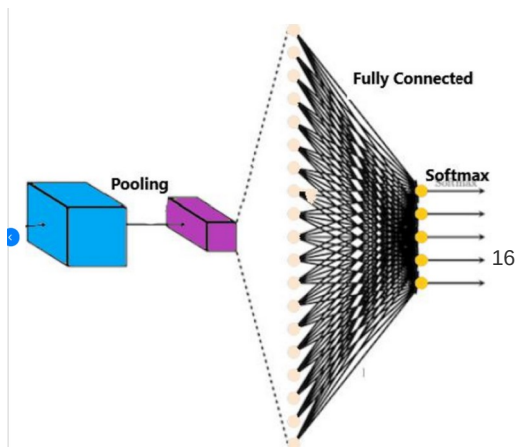


Abbildung 53:
Schematische Darstellung FC-Layer

ReLU-Schicht (Rectified Linear Unit)¹⁷

Ziel:

- Führt Nichtlinearität ein das ist wichtig, damit das Netzwerk komplexe Muster lernen kann.
- Ohne Aktivierungsfunktion wäre das Netzwerk nur eine lineare Abbildung.

Funktionsweise

- Verstärkt relevante Merkmale, unterdrückt irrelevante.



Abbildung 54: Relu-Schicht

Effizient und einfach:

- ReLu ist rechentechnisch sehr günstig
- Keine komplexen Berechnungen
- Viele Neuronen geben 0 aus, das Netzwerk wird „dünner“
- Fördert effizientes Lernen und reduziert Überanpassung

16 https://www.researchgate.net/figure/Example-of-fully-connected-layer_fig4_362857251

17 <https://databasecamp.de/ki/relu>

18 <https://www.ellaberintodefalken.com/2019/10/clasificacion-deep-learning-keras.html>

MaxPooling-Schicht

Ziel:

- Dient dazu, die räumliche Dimension der Feature Maps zu reduzieren, während die relevanten Merkmale erhalten bleiben.
- Diese Reduktion verbessert die Effizienz des Netzwerks, senkt den Rechenaufwand und hilft, Überanpassung zu vermeiden, indem unwichtige Details ausgeblendet werden.

Funktionsweise

- Ein kleines Fenster (z. B. 2×2) wird schrittweise über die Feature-Map verschoben.
- Innerhalb jedes Fensters wird der größte Wert extrahiert und weitergegeben.
- Das Ergebnis ist eine komprimierte Darstellung der ursprünglichen Aktivierung mit geringerer Auflösung, aber konzentriert auf die dominanten Merkmale¹⁹.

Batch Normalization²⁰

- Ziel:
 - Stabilisiert und beschleunigt das Training
 - Es verhindert, dass sich die Verteilung der Eingaben innerhalb des Netzwerks ständig verändert ein Phänomen, dass das Training erschwert. Dadurch bleibt die Datenverteilung konsistenter und das Modell robuster.
- Funktionsweise:
 - Berechnung des Mittelwerts und der Varianz der Ausgabe der vorherigen Schicht.
 - Normalisierung: Subtrahieren des Mittelwerts und Teilen durch die Standardabweichung, so entsteht eine Verteilung mit Mittelwert 0 und Standardabweichung 1.
 - Skalierung und Verschiebung: Damit die Schicht trotzdem flexibel ist, lernt sie zwei Parameter: γ (Skalierung) und β (Verschiebung), die das normalisierte Signal anpassen.
- Effekte:
 - Schnelleres Training: größere Lernraten möglich
 - Weniger empfindlich gegenüber Initialisierung
 - Wirkt leicht Overfitting entgegen

¹⁹ <https://ceosbay.com/2025/01/27/max-pooling-eine-schluesselformel-in-der-bildverarbeitung/>

²⁰ <https://www.geeksforgeeks.org/deep-learning/what-is-batch-normalization-in-deep-learning/>

Softmax

- Ziel:
 - Wandelt rohe Ausgaben (Logits) eines neuronalen Netzes in Wahrscheinlichkeiten um.
 - Wird typischerweise in der letzten Schicht eines Klassifikationsmodells verwendet.
- Eigenschaften:
 - Alle Ausgaben liegen im Bereich $[0,1]$
 - Die Summe aller Ausgaben ergibt genau 1
 - Höhere Logits führen zu höheren Wahrscheinlichkeiten
 - Verstärkt Unterschiede zwischen den Werten (durch die Exponentialfunktion)
 - Beispiel: Wenn ein Modell drei Klassen ausgeben soll und die Logits sind: $[2.0, 1.0, 0.1]$ dann ergibt Softmax etwa: $[0.65, 0.24, 0.11]$ Klasse 1 ist am wahrscheinlichsten²¹

²¹ <https://www.geeksforgeeks.org/deep-learning/the-role-of-softmax-in-neural-networks-detailed-explanation-and-applications/>

3.7 User Story 6 ML-Modell(Einfluss Datenbestand auf Modellqualität)

3.7.1 Erster Versuch

Wir verwenden Datensätze, die ausschließlich Geschwindigkeitsbegrenzung 30er-Schilder oder keine Schilder enthalten. Dabei verarbeiten wir deutlich mehr Bilder „ohne Schilder“ als solche mit 30er-Schilder.

	Label	Count
1	BilderOhneSchilder	3056
2	Geschwindigkeitsbegrenzung_30	314

Abbildung 55: Datenmenge erster Versuch

Wir stellen im Test fest, dass eine stark ungleiche Verteilung der Testbilder pro Kategorie dazu führt, dass die Kategorie mit den meisten Trainingsbildern überproportional häufig erkannt wird. Unser Modell zeigt eine deutliche Verzerrung zugunsten der häufiger vertretenen Klassen.

Trainingsdatensatz

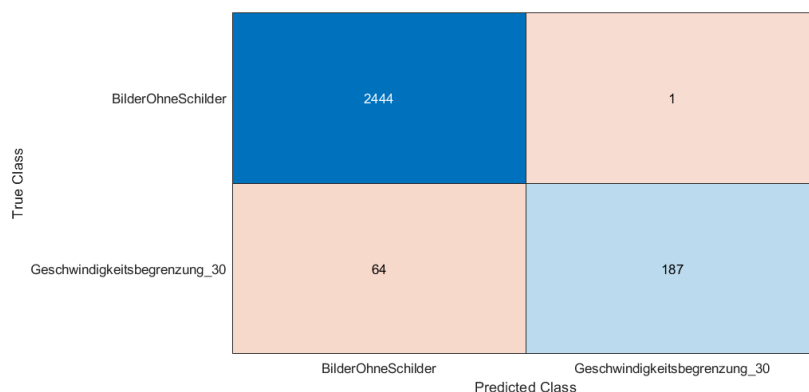


Abbildung 56: Ergebnis Trainingsdatensatz erster Versuch

Testdatensatz

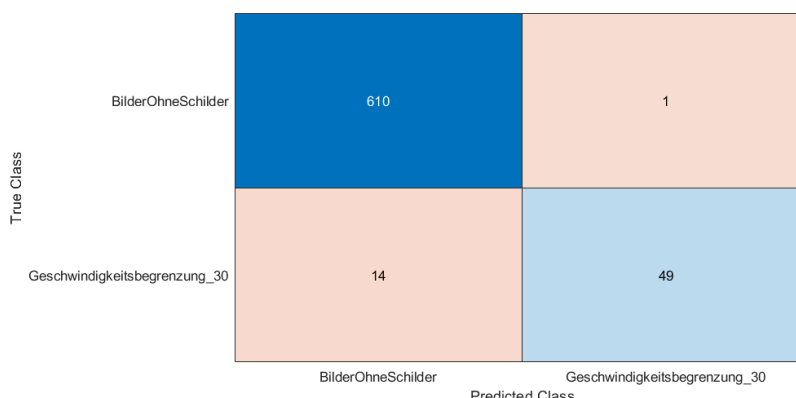


Abbildung 57: Ergebnis Testdatensatz erster Versuch

3.7.2 Zweiter ML Versuch

Wir erhöhen die MiniBatchSize von 64 auf 300. Wir verbessern dadurch zwar die Erkennung der 30er-Schilder im Trainingsdatensatz. Im Testdatensatz stellen wir jedoch keine signifikante Verbesserung fest.

Trainingsdatensatz

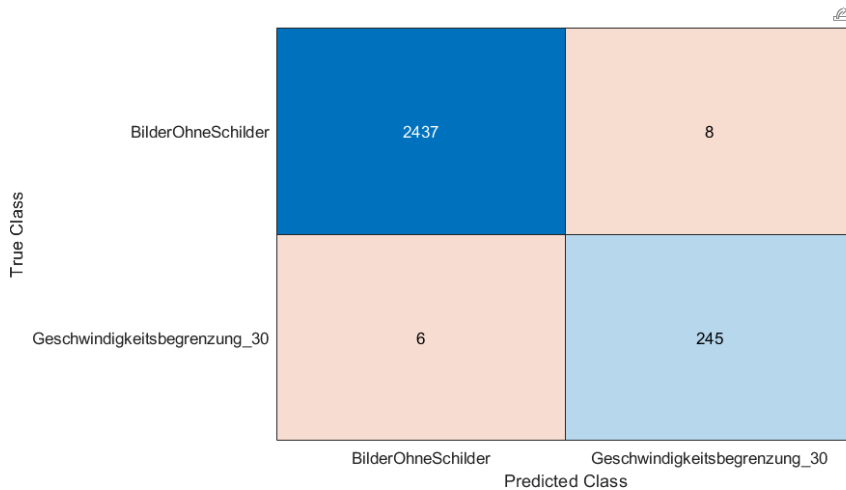


Abbildung 58: Ergebnis Trainingsdatensatz zweiter Versuch

Testdatensatz

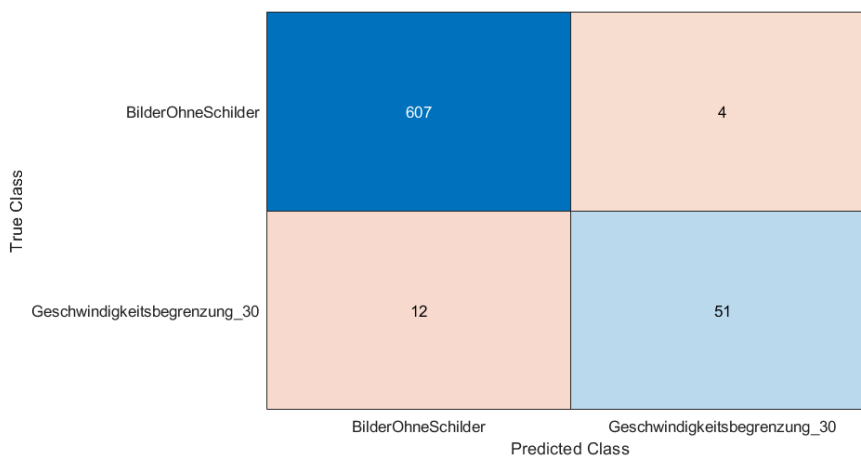


Abbildung 59: Ergebnis Testdatensatz zweiter Versuch

3.7.3 Dritter ML Versuch

Wir gleichen die Anzahl der Trainingsbilder pro Kategorie an und erzielen dadurch eine deutliche Verbesserung der Ergebnisse.

Trainingsdatensatz

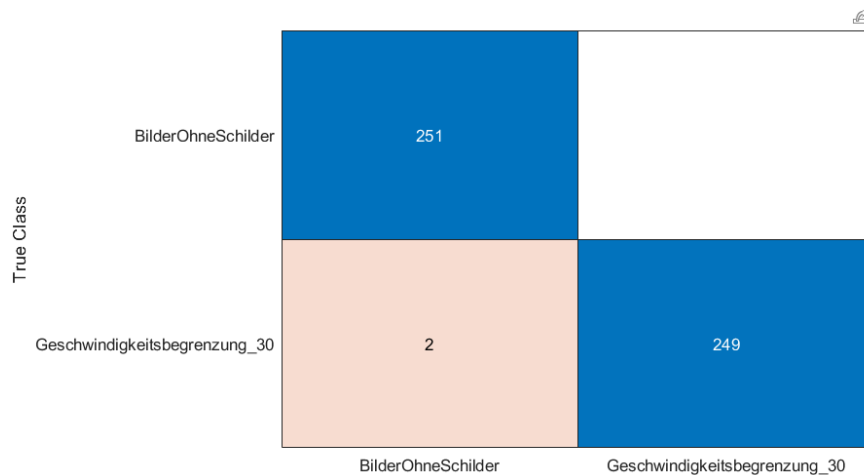


Abbildung 60: Ergebnis Trainingsdatensatz dritter Versuch

Testdatensatz

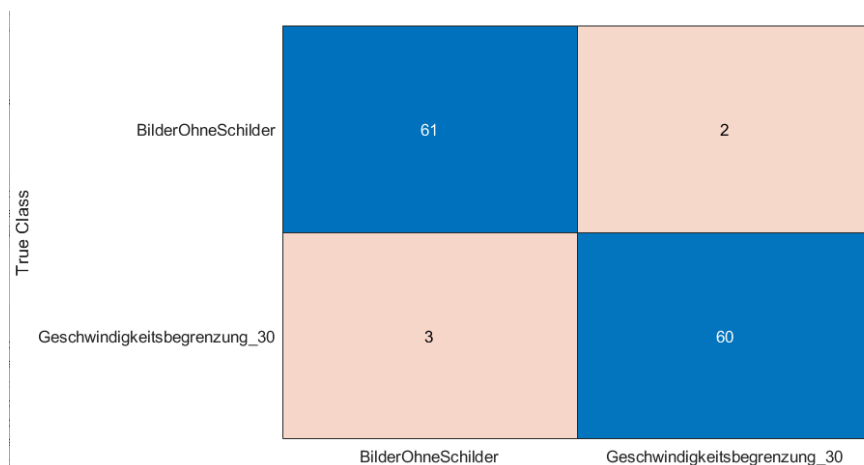


Abbildung 61: Ergebnis Testdatensatz dritter Versuch

3.8 Nutzung des Modells zum Vorsortieren der Trainingsdaten

Beim Training unserer Modelle generieren wir über die Bildvorverarbeitung gezielt Bildausschnitte, immer an den Stellen, an denen Kreise erkannt werden. Aus 300 Bildern pro Schildart entstehen so über 3000 Bildausschnitte. Diese müssen wir manuell in die Kategorien „mit Schild“ und „ohne Schild“ aufteilen, was sich als aufwendiger, monotoner und fehleranfälliger Prozess erweist.

Nachdem wir unserem ML-Modell bei der ersten Schildart beigebracht haben, zwischen Schild und keinem Schild zu unterscheiden, verwenden wir dieses Modell zur automatisierten Sortierung der Bildausschnitte anderer Schildarten. Damit haben wir den Großteil der Bilder sortiert und müssen nur noch wenige Bildausschnitte manuell sortieren.

3.9 Probleme beim Überführen von MATLAB zu OpenMV

Wir übertragen das ML-Modell auf den Mikrocontroller. Dabei treten verschiedene Probleme auf, die sowohl technischer als auch konzeptioneller Natur sind. In Kapitel „3.11.3 Das Problem mit den Layern“ gehen wir auf die technischen Schwierigkeiten genauer ein. In diesem Kapitel erläutern wir die Probleme und überarbeiten das Modell in MATLAB.

3.9.1 Inkompatible Layer(BatchNormalizationLayer)

Das erste Problem, auf das wir stoßen, ist die Inkompatibilität des „BatchNormalization-Layer“ mit OpenMV.

Wir haben zwei Optionen

- Option 1: Layer nachträglich aus dem trainierten Modell entfernen
- Option 2: Wir trainieren direkt ein Modell ohne diesen Layer.

Da der Layer hauptsächlich der Optimierung des Trainings und nicht primär der späteren Erkennung dient und die nachträgliche Entfernung sehr aufwendig und fehleranfällig ist, entscheiden wir uns für Option 2 und bauen den Layer vor dem Training aus.

Durch den Ausbau verschlechtert sich die Genauigkeit deutlich – von etwa 0,25–1 % auf 3–5 %. Um wieder eine vergleichbare Genauigkeit zu erreichen, testen wir verschiedene Ansätze:

Feinjustierung der Learning-Rate

- führt zu keiner Verbesserung

Erhöhung der Epochenanzahl

- führt zu keiner Verbesserung

Anpassung der Batchgröße

- Die Verkleinerung der Batchgröße führt zu einer deutlichen Verbesserung der Genauigkeit.

Im nachfolgenden Bild sind die Ergebnisse von jeweils zehn Lernprozessen mit unterschiedlichen Parameterkonfigurationen zu sehen.

BAtchSize	60	60	60	32	32	32	16
Lernrate/Epoc	0,001/10	0,0005/10	0,0005/15	0,0005/15	0,0005/10	0,001/10	0,001/10
min	1,03%	0,96%	1,39%	1,39%	0,73%	0,73%	0,60%
max	5,43%	4,08%	5,04%	3,64%	4,74%	3,14%	2,55%
median	2,75%	2,67%	2,75%	2,52%	2,78%	1,56%	1,49%

Abbildung 62: Parameter nach entfernen Batch-layer

3.9.2 Modellgröße

Nach unserer Anpassung erhalten wir eine Fehlermeldung, die darauf hindeutet, dass das Modell zu groß ist. Das Modell ist 1,4 MB groß und sollte auf den 4 MB Speicher des Mikrocontrollers passen. Trotzdem verkleinern wir es, um den Fehler auszuschließen. Wir reduzieren die Bildgröße schrittweise auf 48×48 Pixel. Dadurch erreicht das Modell eine Größe von nur noch 88 kB bei weiterhin guter Genauigkeit.

Testdatensatz

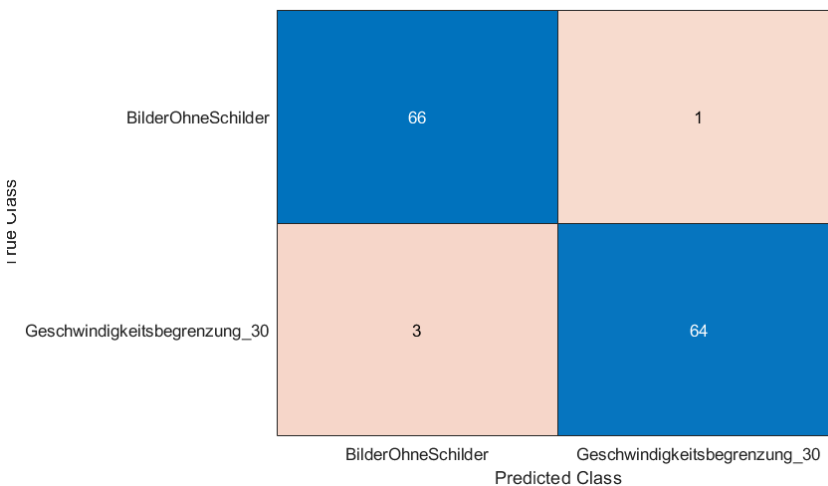


Abbildung 63: Ergebnis Testdatensatz Modellgröße reduzieren

3.9.3 Probleme mit FC-Layer

Da wir weiterhin denselben Fehler erhalten, vermuten wir, dass zusätzliche inkompatible Layer die Ursache sind. Deshalb analysieren wir die mit OpenMV mitgelieferten Beispiel-ML-Modelle²² und prüfen, welche Layerarten dort verwendet werden. Dabei stellen wir fest, dass in keinem dieser Modelle ein Fully-Connected-Layer enthalten ist.

Daraufhin entfernen wir diesen Layer testweise aus unserem Modell. Da das System anschließend fehlerfrei auf dem Mikrocontroller läuft, entwickeln und trainieren wir ein eigenes Modell, das vollständig ohne Fully-Connected-Layer auskommt. Zur Umsetzung ergänzen wir das Modell um die folgenden zwei Layer.

```
convolution2dLayer(1, 2, "Name", "conv_3") % ersetzt das Fully Connected Layer  
globalAveragePooling2dLayer("Name", "gap")
```

Abbildung 64: MATLAB-Code FC-Layer entfernen

Der **Conv_3-Layer** reduziert die Anzahl der Feature-Maps auf die gewünschte Kanalzahl → in unserem Fall auf zwei Klassen. An jeder Bildposition kombiniert er die zuvor vorhandenen 16 Feature-Kanäle zu genau zwei neuen Kanälen. Auf diese Weise entstehen zwei sogenannte Klassen-Karten, die für jeden Ort im Bild den Grad der

²² <https://docs.openmv.io/develop/cmodules.html>

Zugehörigkeit zu Klasse 1 bzw. Klasse 2 darstellen. Dadurch werden aus einer Vielzahl an Merkmalen unmittelbar klassenbezogene Signale generiert.

Der **Global-Average-Pooling GAP-Layer** reduziert anschließend jede dieser Klassen-Karten auf einen einzelnen Wert, indem er den Mittelwert über alle räumlichen Positionen ($H \times W$) berechnet. Somit entstehen zwei kompakte Scores → je einer pro Klasse, unabhängig davon, an welcher Position im Bild die entsprechenden Merkmale auftreten.

Ohne den **Conv_3-Layer** würde der **GAP-Layer** die ursprünglichen 16 Kanäle auf 16 Scores mitteln, sodass eine weitere Schicht erforderlich wäre, um diese auf zwei Klassen zu reduzieren. Ohne den **GAP-Layer** läge das Ergebnis in Form einer $12 \times 12 \times 2$ -Matrix vor, für die anschließende Softmax-Funktion wird jedoch eine 1×2 -Matrix benötigt.

3.10 Bildvorverarbeitung von MATLAB auf OpenMV Board bringen

Um die Bildvorverarbeitung mit gleicher Funktionalität auch auf dem Cam-Board nutzen zu können, suchen wir einen Weg, um den Code zu konvertieren und in Micropython nutzbar zu machen.

3.10.1 3 Wege zum Ziel – die Alternativen

Nach einer Recherche der MATLAB und OpenMV Dokumentation stellen wir fest, dass es 3 Wege gibt, die vorhandene Bildvorverarbeitung von MATLAB auf das OpenMV Board zu bringen:

1. Exportieren des MATLAB Bildvorverarbeitungs-Codes mithilfe des codegen Befehls des MATLAB Coder in C/C++ Code und anschließend die OpenMV MicroPython-Firmware inklusive des generierten Codes komplett neu kompilieren²³
2. Ebenfalls den MATLAB Code als C/C++ Code exportieren, diesen Code dann aber mithilfe eines Tools von OpenMV als nativen Maschinencode in eine .mpy Datei verlinken, die dynamisch in MicroPython geladen werden kann²⁴
3. Äquivalente Bildverarbeitungsfunktionen in den OpenMV Bibliotheken finden und die Bildvorverarbeitung noch einmal in MicroPython implementieren

Mit 2 Monaten verbleibender Projektzeit entscheiden wir uns – aufgrund mangelnder Erfahrung beim kompilieren nativen Codes – *dagegen*, Option 1 auszuprobieren.

Wir prüfen die 2. Variante, entscheiden uns am Ende aber für die 3. Variante. Dennoch wollen wir hier einmal den Ansatz der zweiten Variante zeigen.

3.10.2 Ansatz: MATLAB's Code-Generator

Die Dokumentation von MATLAB's Image Processing Toolbox wirbt mit einem Code-Generator für C-Code²⁵.

Um die oben beschriebenen zweite Variante über eine .mpy Datei zu testen, schreiben wir den Code der Bildvorverarbeitung um. Das Skript arbeitet aktuell direkt mit Bilddateien auf dem PC, auf welchem MATLAB ausgeführt wurde. Für den C-Code benötigen wir aber eine Funktion, welche als Parameter die Pixelwerte des RGB Bildes entgegen nimmt.

Eine Minimaldatei zum reproduzieren lässt sich von [Github](#) herunterladen.

²³ <https://docs.openmv.io/develop/cmodules.html>

²⁴ <https://docs.openmv.io/develop/natmod.html>

²⁵ <https://de.mathworks.com/help/images/code-generation-and-gpu-support.html>

Mit der umgeschriebenen Funktion führen wir folgende Befehle im Command Window von MATLAB aus:

```
cfg = coder.config('lib');  
cfg.TargetLang = 'C';  
codegen -config cfg bildvorverarbeitung.m -args {zeros(1280,720,3,'uint8')}  
-report
```

Daraufhin wird der C-Code in den Ordner ./codegen/lib/bildvorverarbeitung generiert.

In diesem Ordner liegen die Dateien ./entrypoint/main.c und entrypoint/main.h, welche die Einstiegsfunktion `int main(int argc, char **argv)` enthalten.

Um diesen generierten Code nun in OpenMV nutzen zu können, müssen laut der [Dokumentation](#) folgende Schritte befolgt werden:

1. Es muss in der main.c und main.h eine Funktion `mpy_init` als Entrypoint deklariert werden
2. Die Datei `py/dynruntime.h` aus dem OpenMV Repository über eine Include Anweisung eingebunden werden
3. Alle zu kompilierenden Dateien müssen zusammen mit der Zielarchitektur in einem Makefile aufgelistet werden, die Dokumentation von OpenMV hält dazu ein Beispiel bereit

An dieser Stelle entscheiden wir uns dann aufgrund von Zeitmangel und mangelnder Kenntnis über das Tool „make“ dazu, dass es schneller ist, den vorhandenen MATLAB Bilderkennungscode nochmal in OpenMV umzusetzen.

3.10.3 Bildvorverarbeitung – der MicroPython-Weg

Um also die Bildvorverarbeitung noch einmal in MicroPython umzusetzen, müssen wir Bildverarbeitungsalgorithmen finden, die eine äquivalente Funktion zu den Algorithmen von MATLAB bieten. Da OpenMV eine ganze Reihe eigener Bildverarbeitungsfunktionen und -algorithmen bietet, ist die Funktionsauswahl nicht kompliziert.

Der Eingabeparameter der Bildvorverarbeitung ist ein einzelnes Bild mit der Auflösung von 1280 x 720 Pixeln pro Aufruf. Der Ausgabeparameter ist ein 200x200 Pixel großer Bildausschnitt, der auf 48x48 Pixel herunter-skaliert wird. Die Auflösung des Bildausschnitts haben wir so klein gewählt, damit wir zuerst mit einem möglichst kleinen Machine Learning Modell testen können, ob es in den RAM des OpenMV-Boards passt.

Hier eine beispielhafte Gegenüberstellung einiger gekürzter Funktionssignaturen von MATLAB und MicroPython in Pseudocode:

MATLAB

```
rgb2gray(rgbImage)
=> grayscaleImage

imfindcircles(image, radius: [int, int],
'Sensitivity', 0.90)
=> [centers, radii]

imgaussfilt(imageToFilter, sigma: int,
„FilterSize“, 3)
=> filteredImage

edge(image, method = „Canny“)
=> binaryImageWithWhiteOnEdge

imbinarize(grayscaleImage, threshold: int)
=> binaryImage
```

OpenMV / MicroPython

```
image.to_grayscale()
=> Image

image.find_circles(threshold = 2000, r_min: int,
r_max: int)
=> List[circle]

image.gaussian(kernelSize = 3)
=> Image

image.find_edges(edge_type = „image.EDGE_CANNY“)
=> void (mutates object)

image.binary(thresholds: List[Tuple[int,int]])
=> binaryImage
```

3.10.4 Gegenüberstellung des Zeitaufwandes

Die Recherche für den Export der Bildvorverarbeitung von MATLAB über C nach MicroPython dauerte 2 Abende.

Das händische Suchen von Äquivalenzfunktionen und neu-implementieren in MicroPython mit OpenMV Bildverarbeitungs-Algorithmen war an einem Abend erledigt.

Dies rechtfertigt in unseren Augen den manuellen Aufwand.

3.11 User Story 7 – ML-Modell von MATLAB auf OpenMV Board bringen

Unser Ziel laut dem Vision-Statement ist es, ein Machine Learning Model in MATLAB trainieren zu können und auf dem OpenMV Board auszuführen. Dazu muss das fertig Trainierte Modell aus MATLAB exportiert und für die OpenMV Firmware nutzbar gemacht werden.

3.11.1 Die Möglichkeiten für Export und Import

Auch an dieser Stelle finden wir in der Dokumentation von MATLAB 3 Wege, um ein Machine Learning-Model aus der Machine Learning Toolbox zu exportieren:

1. Mit dem MATLAB Coder C/C++ Code generieren und für OpenMV nutzbar machen.
2. Mithilfe des Befehls `exportONNXNetwork`:
 - 2.1 Eine Datei im **Open Neural Network Exchange**²⁶ Format exportieren
 - 2.2 In ein TensorFlow Model importieren
 - 2.3 Mit Python Quantisieren und in eine .tflite Datei konvertieren
3. Mithilfe des Befehls `exportNetworkToTensorFlow`:
 - 3.1 Ein TensorFlow Keras Modell exportieren
 - 3.2 Mit Python Quantisieren und in eine .tflite Datei konvertieren

Variante 1 schließen wir – wie bei der Bildvorverarbeitung auch – aufgrund mangelnder Zeit und fehlender Erfahrung, aus. Außerdem besteht das Risiko, dass ein Machine Learning Modell aus generiertem C-Code nicht mit den geringen Ressourcen eines Mikrocontrollers auskommt und die investierte Zeit zu keinem Ergebnis führt.

Variante 2 entfällt durch einige Versionsprobleme von Python und TensorFlow im Zusammenhang mit der deprecation von `onnx_tf`²⁷. Außerdem stellt sich der problematische Schritt 2.2 als überflüssig heraus, da MATLAB auch direkt ein TensorFlow Modell exportieren kann.

²⁶ <https://de.wikipedia.org/wiki/ONNX>

²⁷ <https://github.com/onnx/onnx-tensorflow>

3.11.2 Mit TensorFlow eine .tflite Datei erzeugen

Der Befehl `exportNetworkToTensorFlow` ermöglicht es uns, das MATLAB ML-Modell in mehreren Python-Skripts mit einer separaten Datei für die Gewichtungen zu exportieren. Die Scripts enthalten eine Modelldefinition der Layer, geschrieben mit Keras aus der TensorFlow Bibliothek. Ebenfalls enthalten ist eine Funktion, um das Modell inklusive Gewichtung zu laden, damit es zu verwendet werden kann.

Laut der TensorFlow Dokumentation²⁸ kann die OpenMV Firmware mit TensorFlow Lite umgehen. Dazu muss das Keras Modell über die Python Klasse `tf.lite.TFLiteConverter` in eine .tflite Datei konvertiert und dabei auch Quantisiert werden.

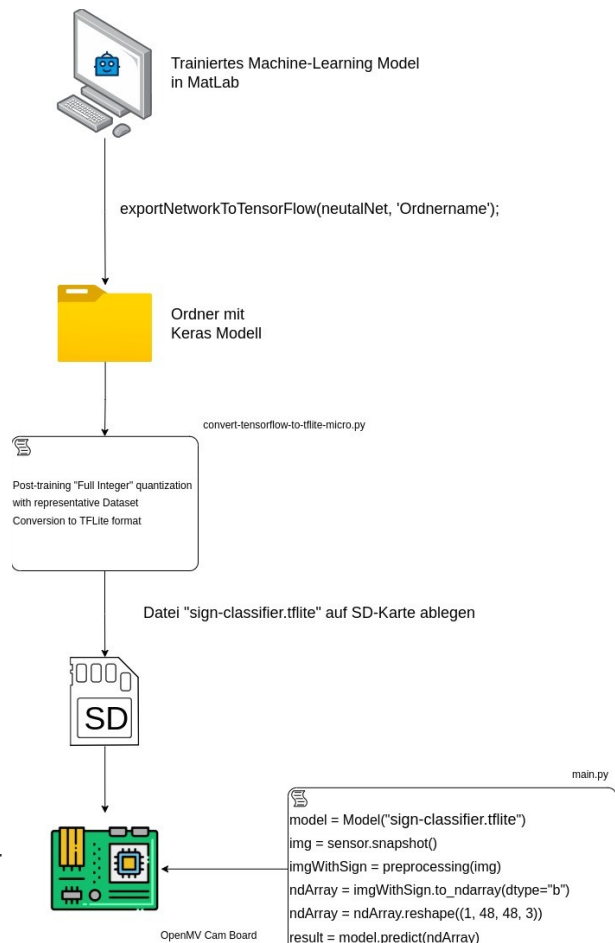
Die Quantisierung besteht in unserem Fall daraus, die präzise, aber speicher-hungrige 32-bit Fließkommaberechnung in einem Machine Learning Modell in eine Ressourcenschonende 8-bit Ganzzahlberechnung umzuwandeln.

Bei diesem Vorgang geht durch Rundungsfehler natürlich die Genauigkeit des Modells verloren. Deshalb verlangt TensorFlow für die Umwandlung ein repräsentatives Set von Daten, um mit statistischen Berechnungen der typischerweise auftretenden Wertebereiche den Genauigkeitsverlust zu minimieren²⁹. Dieses Thema wollen wir nicht weiter vertiefen, da es den Rahmen sprengen würde.

Um die Konvertierung nachzustellen, können sie das Python-Skript „convert - tensorflow-to-tflite.py“ unter diesem [GitHub Link](#) nutzen. Um die korrekten Bibliotheken mit den richtigen Versionen zu installieren, empfiehlt es sich, Python 3.11.11 zu installieren, ein Python venv (Virtual Environment) zu öffnen und mit „pip install -r python-requirements.txt“ zu installieren.

Die Befehle mitsamt der Schrittreihenfolge von Matlab bis zur .tflite sind auch noch einmal in der Zugehörigen [README](#) erklärt.

Läuft das Skript erfolgreich durch, können wir die erzeugte .tflite Datei auf der SD-Karte des Cam-Boards ablegen und mit



28 <https://docs.openmv.io/library/omv.ml.html>

29 https://www.tensorflow.org/api_docs/python/tf/lite/TFLiteConverter suche nach „representative_dataset“

```
model = ml.Model("/sdcard/models/sign-classifier.tflite")  
model.predict([image_array])
```

laden, sowie ausführen. Je nachdem, mit welchen Layern wir das Modell erstellen, werden wir mit der kryptischen Fehlermeldung „Failed to allocate tensors“ darüber informiert, dass unsere Datei nicht ausführbar ist.

3.11.3 Das Problem mit den Layern

Nach einigen Experimenten mit den verschiedenen Layerkompositionen stellen wir fest, dass OpenMV nicht alle Layer unterstützt und bei der Konvertierung von MATLAB zu TensorFlow Lite sehr darauf geachtet werden muss, wie das Modell aufgebaut ist.

MATLAB arbeitet mit der Tensor-Layout-Convention HWCN, also **H**eight, **W**idth, **C**hannel und **N**umber (Batchsize), während TensorFlow mit NCHW Arbeitet. Beim exportieren von MATLAB zu Tensorflow wird HWCN zu NCHW überführt.

Allerdings kommt es dabei vor dem FullyConnected Layer zu der rechts stehenden Anomalie³⁰. Im Normalfall erwarten wir hier einen einzelnen Strang mit der Verkettung der Layer.

Der Parallelpfad ist aber nicht das einzige Problem. OpenMV unterstützt TensorFlow Lite for Microcontrollers^{31 32}, nicht das volle TensorFlow Lite.

Es lässt sich keine vollständige Liste von Kompatiblen Layern im Internet finden, allerdings tritt der Fehler „Failed to Allocate Tensors“ immer dann auf, wenn die Layer „Shape“, „StridedSlice“, „Pack“, „Reshape“ oder „FullyConnected / Dense“ verwendet werden.

MATLABS classificationLayer, welches nach der Softmax Funktion aufgerufen wird, wird bei der Konvertierung in TensorFlow Lite ebenfalls durch ein Reshape Layer ersetzt.

Das FullyConnected Layer nimmt als Eingang einen 1D-Vektor. Bei der Suche einer Lösung finden wir auf [reddit](#) die Erklärung, dass das FullyConnected Layer durch ein GlobalAveragePooling2D Layer und ein Conv2D Layer mit einer Kernel Größe von 1x1 ersetzt werden kann. Ein [Beitrag auf Medium](#) erklärt außerdem, dass auch ein Conv2D Layer mit der gleichen Kernel Größe des Inputs verwendet werden kann. Wenn also ein 200x200 Pixel großes Bild als Input dient, sollte der Kernel 200x200 groß sein.

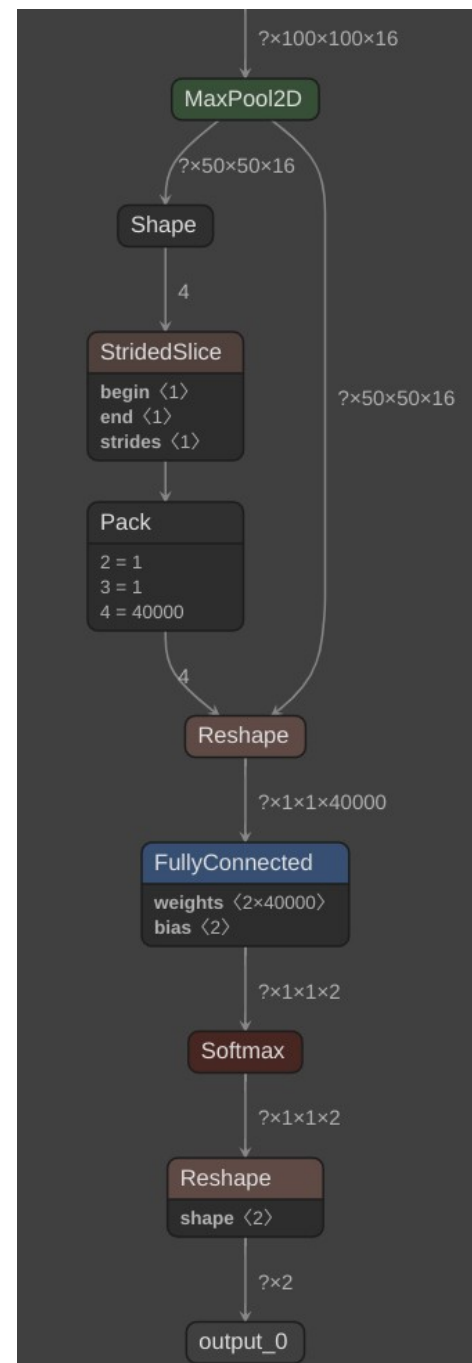


Abbildung 65: Beispielhaftes Modell - nicht auf OpenMV lauffähig

30 Screenshot des Modells aus <https://netron.app>

31 <https://docs.openmv.io/library/omv.tf.html>

32 <https://ai.google.dev/edge/litert/microcontrollers/overview>

Wir prüfen also die Lösung mit GlobalAveragePooling2D und Conv2D mit der Kernel Größe 1x1. Außerdem verwenden wir in Matlab die trainnet-, anstatt der trainNetwork-Funktion, um das Modell zu trainieren, wodurch wir den classificationLayer entfernen können.

Anschließend konvertieren wir das Modell erneut in eine .tflite Datei und schauen uns dieses in Netron an.

Wie wir auf dem Graphen erkennen können, ergibt diese Lösung ein sequenzielles Modell. Dieses lässt sich über den MicroPython Code

```
model = ml.Model("/sdcard/models/sign-classifier.tflite")
```

erfolgreich Laden und mit

```
output = model.predict([image_array])
scores = output[0][0][0][0]
print(scores)
```

bekommen wir eine Ausgabe die der Folgenden ähnelt:

```
array([0.5, 0.5], dtype=float32)
```

Der lange Index-Zugriff output[0][0][0][0] ist nötig, da der Output des Modells einem 4D-Tensor der Größe 1x1x1x2 entspricht und somit im Code wie ein 4 Dimensionales Array anzusprechen ist.

Die beiden Werte von 0.5 entsprechen der Wahrscheinlichkeit der beiden möglichen Klassifizierungen, der erste Wert „Bild mit Schild“ und der zweite Wert „Bild ohne Schild“. Diese Beispielwerte stammen von einem schwarzen Bild mit der Abdeckkappe auf der Linse, wodurch die Werte von 0.5 Zustande kommen.

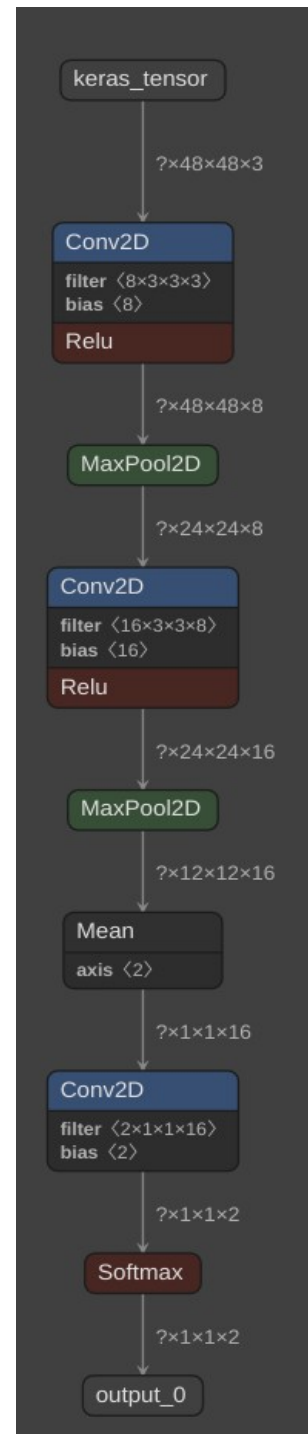


Abbildung 66: Mit OpenMV kompatibles Modell

4 Ergebnis

Das Modell ist nun auf dem OpenMV Cam H7 Plus lauffähig.

Im folgenden sehen wir das Ergebnis des Produktiv betriebenen Modells.

Der erste Testfall zeigt die Klassifikation eines Bildes, auf dem zwar ein Kreis, aber kein Schild zu sehen ist:



Abbildung 67: Ergebnistest -
ohne Schild



Abbildung 68:
Ergebnistest -
Kreis gefunden

```
Klassifikation: array([0.304688, 0.695313], dtype=float32)
```

Abbildung 69: Ergebnistest - Klassifikation "Bild ohne Schild"

Der Test zeigt, dass die Bildvorverarbeitung auf dem Bild einen Kreis erkannt und ausgeschnitten hat. Das Machine Learning Modell klassifiziert den Bildausschnitt zu gerundet 70% als „Bild ohne Schild“.

Der zweite Testfall zeigt die Klassifikation eines Bildes mit Schild:



Abbildung 70: Ergebnistest
- mit Schild



Abbildung 71:
Ergebnistest -
Schild gefunden

```
Klassifikation: array([0.984375, 0.015625], dtype=float32)
```

Abbildung 72: Ergebnistest - Klassifikation "Bild mit Schild"

Auch hier wird das Schild als Kreis erkannt und Ausgeschnitten. Das Modell Klassifiziert den Bildausschnitt zu 98% als „Bild mit Schild“.

4.1 Persönlicher Erfahrungsgewinn / Lessons Learned

4.1.1 Philip

Ich habe vor diesem Projekt noch keinerlei Versuche unternommen, selbst ein Machine Learning Modell zu trainieren. Ich betrachte den Hype um KI-Modelle sogar etwas kritisch, weil ich auf die Leistungsfähigkeit von Maschinen vertraue, die speziell für eine Problemstellung konstruiert wurden, anstatt einer einzigen KI vielfältige Aufgaben anzuvertrauen.

Durch dieses Projekt konnte ich sehen, dass ein *gezielt* trainiertes Modell zwar sehr Aufwändig ist, als Unterstützung für den Menschen aber sehr Hilfreich sein kann.

Ohne groß in die Tiefe gegangen zu sein, scheint mir die Mathematik hinter den Modellen sehr Interessant. Vielleicht werde ich in Zukunft mal ein paar Testmodelle direkt mit TensorFlow und Keras aufsetzen und mir die Theorie hinter neuronalen Netzen ansehen. Eventuell probiere ich aber auch mal PyTorch aus.

Das Projektmanagement und die Zusammenarbeit mit jemandem zusammen zu Arbeiten, der in einer ganz anderen Branche arbeitet, war sehr Aufschlussreich.

Was ich im nächsten Projekt anders machen würde:

- Falls ich noch einmal ein Gantt Chart brauche, würde ich mir im voraus die Bedienung von ProjectLibre genauer ansehen. Gerade die Baselines bzw. Basispläne scheinen mir wie ein Muss. Da ich in meiner Berufspraxis noch kein Gantt Chart selbst erstellen musste, wusste ich nicht, dass Baselines existieren.
- Den Weg zum deployment auf dem Endgerät würde ich möglichst früh im Projektverlauf ansehen. Wir sind zu Beginn davon ausgegangen, dass der Schritt von MATLAB auf das OpenMV Board viel leichter zu handhaben wäre. Im Endeffekt mussten wir viele Schritte mehrfach wiederholen, weil wir einfach ein Modell trainiert haben, was auf der OpenMV Firmware nicht ausführbar war.
- Gehäuse und Halterungen werden konstruiert, bevor Testdaten gesammelt werden.
- Ich muss mir noch ein besseres Tool zum Zeichnen von allgemeinen Diagrammen und UML-Diagrammen suchen. Ich bin weder mit Modelio, noch mit PlantUML wirklich zufrieden. Die vorhandenen Icons von DrawIO haben mir auch nicht alle so gut gefallen. Vielleicht muss ich einfach selbst die Symbolik zeichnen.

Alles in allem bin ich mit dem Ergebnis von nur zwei Personen sehr Zufrieden. Die Arbeitsschritte von MATLAB bis aufs Board sind nun bekannt und das Modell ist auf dem Microcontroller lauffähig. Alles weitere, wie die Erkennungswahrscheinlichkeit und Modellperformance, ist Fleißarbeit.

4.1.2 Nico

Zu Beginn des Projekts ging ich davon aus, dass insbesondere die Themen Bildvorverarbeitung und die Entwicklung eines Machine-Learning-Modells große Herausforderungen darstellen würden. Ich rechnete damit, häufig auf Schwierigkeiten zu stoßen und auf die Unterstützung des Dozenten angewiesen zu sein. Tatsächlich erwiesen sich diese Bereiche als sehr zeitintensiv und komplex, und es traten immer wieder neue Hindernisse auf. Dennoch gelang es uns, diese eigenständig zu überwinden. Durch diese Erfahrung erkenne ich, dass es heute durchaus möglich ist, sich die notwendigen Kenntnisse im Bereich Machine Learning eigenständig anzueignen, selbst mit nur wenigen Vorkenntnissen.

Die Arbeit mit MATLAB hat mir ein Verständnis für die Grundlagen des maschinellen Lernens vermittelt. Besonders motivierend war es, die technischen Herausforderungen zu lösen und ein funktionierendes Modell zu entwickeln. Diese positive Erfahrung hat in mir das Interesse geweckt, auch zukünftig hobbymäßig Projekte in diesem Bereich zu verfolgen.

Im Rahmen der Zusammenarbeit konnte ich zudem neue Themenfelder wie GitHub und Python kennenlernen.

Was ich im nächsten Projekt anders machen würde:

- **Fokussierung auf ein einzelnes Schild:** Zu Projektbeginn würde ich mich ausschließlich auf ein spezifisches Schild konzentrieren, sowohl hinsichtlich der Bildaufnahme und Vorverarbeitung als auch bei der Modellerstellung. Dies würde ermöglichen, den Transfer auf den Mikrocontroller frühzeitig zu testen, da sich hier die größten Herausforderungen gezeigt haben.
- **Effizienz in der Bildvorverarbeitung:** Die Bildvorverarbeitung in MATLAB würde ich nur so weit optimieren, wie es für das Training des Modells erforderlich ist. Aufwendige Parameteroptimierungen würde ich vermeiden, da die Bildverarbeitung in OpenMV ohnehin neu implementiert werden muss.
- **Zwischenschritt zur Validierung des Modells nach Quantisierung:** Ich würde einen zusätzlichen Validierungsschritt einbauen, um zu prüfen, ob das Modell nach der Quantisierung weiterhin zufriedenstellende Ergebnisse liefert. Dies würde helfen, mögliche Fehlerquellen auf dem Mikrocontroller besser zu identifizieren und gezielt zu beheben.

Mit dem Projektergebnis bin ich sehr zufrieden. Berücksichtigt man, dass wir beide zu Projektbeginn nur geringe oder gar keine Vorkenntnisse in den relevanten Themenbereichen wie Bildvorverarbeitung, Machine Learning, OpenMV und weiteren hatten, ist das Erreichte umso bemerkenswerter. Im Verlauf des Projekts konnten wir zahlreiche Grundlagen schaffen, auf denen zukünftige Arbeiten aufbauen können. Dazu zählen unter anderem:

- Eine umfangreiche Sammlung von Beispielbildern

- Ein funktionsfähiges Gehäuse für die Hardware
- Verschiedene Methoden zur Bildvorverarbeitung inklusive unterstützender Funktionen
- Erkenntnisse über kompatible Layer-Strukturen
- Dokumentierte Probleme beim Modelltransfer
- Ein Tool zur Modellübersetzung
- Ein lauffähiges Beispiel auf dem Mikrocontroller

Diese Ergebnisse bilden eine solide Basis für weiterführende Entwicklungen.

5 Ausblick

Als Ausblick für ein Anschlussprojekt möchten wir folgendes empfehlen:

- Die Performance des Machine Learning Modells auf dem OpenMV Board ist steigerbar. Dazu ist zu prüfen, ob die Auflösung von 1280 * 720 Pixeln eventuell zu groß ist. Außerdem kann der Micropython Code „eingefroren“ werden³³, wodurch die Ausführungszeit gesteigert wird.
- Die Bildvorverarbeitung kann um die Erkennung für mehr Schildarten erweitert werden. Anschließend kann pro Schildform ein weiteres Machine Learning Modell trainiert werden.
- Die Genauigkeit der Klassifizierung kann verbessert werden. Dazu sollten noch einmal die Auflösungen der genutzten Trainingsbildern von MATLAB mit den aufgenommenen Bildern von OpenMV verglichen werden.
Es muss sicher gestellt sein, dass der Bildausschnitt, der an das Modell übergeben wird, die gleiche Auflösung besitzt wie beim Training des Modells. Außerdem kann der representative Datensatz für die Quantisierung auf Sinnhaftigkeit kontrolliert werden.
- Auf dem Displaymodul können die erkannten Bilder inklusive der Wahrscheinlichkeit der Klassifizierung angezeigt werden. Dies haben wir aus Zeitgründen nicht implementiert.
- Die Entwicklung eines Tools, mit dem automatisiert überprüft werden kann, ob das Modell nach der Quantisierung weiterhin eine ähnliche Genauigkeit erzielt, wie vor der Quantisierung.

33 <https://docs.openmv.io/openmvcam/tutorial/production.html>

Abbildungsverzeichnis

Abbildung 1: Work Breakdown Stucture.....	7
Abbildung 2: Initialer Projektplan.....	8
Abbildung 3: Verlängerung der Projektinitialisierung.....	9
Abbildung 4: Vorziehen von Gehäusedesign.....	9
Abbildung 5: Verlängerung „Testdatensätze aufnehmen“ und Unterteilung Meilenstein....	10
Abbildung 6: Modell auf MC spielen (Vorher).....	10
Abbildung 7: Modell auf MC spielen (Nacher).....	10
Abbildung 8: Fehlerbeseitigung.....	11
Abbildung 9: Prinzipaufbau des Mikrocontroller-Board inklusive Peripherie.....	14
Abbildung 10: Einzelkomponenten Vorderseite.....	14
Abbildung 11: Einzelkomponenten Rückseite.....	14
Abbildung 12: Leisten 1.....	15
Abbildung 13: Leisten 2.....	15
Abbildung 14: Leisten 3.....	15
Abbildung 15: Arbeitsplatz zum Löten.....	15
Abbildung 16: Arbeitsplatz Nahaufnahme.....	15
Abbildung 17: Lötzinn und Entlötlitze.....	16
Abbildung 18: Lotflussmittel ULF-10.....	16
Abbildung 19: Isopropylalkohol.....	16
Abbildung 20: Reinigungsstäbchen.....	16
Abbildung 21: Verlötete Buchsenleiste rechts.....	17
Abbildung 22: Verlötete Buchsenleiste links.....	17
Abbildung 23: OpenMV µC Board mit WIFI und Display Modul.....	17
Abbildung 24: Connect.....	18
Abbildung 25: Konzept des Trainings.....	19
Abbildung 26: Aufruf des Fotoapparats über die Adresse http://192.168.0.101:8080	20
Abbildung 27: Erfolgs- und Fehlermeldung bei der Bildaufnahme.....	20
Abbildung 28: Gehäuse in Einzelnteilen.....	22
Abbildung 29: Gehäuse montiert von links.....	22
Abbildung 30: Gehäuse montiert von rechts.....	22
Abbildung 31: MATLAB – Bilder einlesen.....	26
Abbildung 32: MATLAB – Erkennung runde Schilder.....	27
Abbildung 33: Kreiserkennung – Farb- und Grau-bild.....	28
Abbildung 34: Kreiserkennung – Rauschunterdrückung und Binärbild.....	28
Abbildung 35: Kreiserkennung – Sensitivity.....	28
Abbildung 36: Beispielbild – Hohe Sensitivität.....	29
Abbildung 37: MATLAB-Code Sensitivität.....	29
Abbildung 38: Erkennung rote runde Schilder.....	29
Abbildung 39: Blau Schilder.....	30
Abbildung 40: Blaue Schilder Fehlerkennung.....	30
Abbildung 41: Parametervergleich.....	30
Abbildung 42: Parametervergleich 2.....	31
Abbildung 43: Parametervergleich 3 erfolgreich.....	31
Abbildung 44: MATLAB-Code 3eck Erkennung.....	32
Abbildung 45: MATLAB-Dreieckerkennung.....	32
Abbildung 46: 3eck Erkennung.....	32
Abbildung 47: MATLAB_Code Bild Invertierung.....	33
Abbildung 48: Bild Invertierung.....	33

Abbildung 49: MATLAB-Code Toleranz blaue Vierecke.....	33
Abbildung 50: blaue Vierecke.....	33
Abbildung 51: Total learnables.....	36
Abbildung 52: Layeransicht MALAB.....	36
Abbildung 53: Schematische Darstellung FC-Layer.....	38
Abbildung 54: Relu-Schicht.....	38
Abbildung 55: Datenmenge erster Versuch.....	41
Abbildung 56: Ergebnis Trainingsdatensatz erster Versuch.....	41
Abbildung 57: Ergebnis Testdatensatz erster Versuch.....	41
Abbildung 58: Ergebnis Trainingsdatensatz zweiter Versuch.....	42
Abbildung 59: Ergebnis Testdatensatz zweiter Versuch.....	42
Abbildung 60: Ergebnis Trainingsdatensatz dritter Versuch.....	43
Abbildung 61: Ergebnis Testdatensatz dritter Versuch.....	43
Abbildung 62: Parameter nach entfernen Batch-layer.....	44
Abbildung 63: Ergebnis Testdatensatz Modellgröße reduzieren.....	45
Abbildung 64: MATLAB-Code FC-Layer entfernen.....	45
Abbildung 65: Beispielhaftes Modell - nicht auf OpenMV lauffähig.....	53
Abbildung 66: Mit OpenMV kompatibles Modell.....	54
Abbildung 67: Ergebnistest - ohne Schild.....	55
Abbildung 68: Ergebnistest - Kreis gefunden.....	55
Abbildung 69: Ergebnistest - Klassifikation "Bild ohne Schild".....	55
Abbildung 70: Ergebnistest - mit Schild.....	55
Abbildung 71: Ergebnistest - Schild gefunden.....	55
Abbildung 72: Ergebnistest - Klassifikation "Bild mit Schild".....	55